Quadratic-Time Balanced Minimum Evolution Discrete Gradients, Column-Tiled SIMT Parallelism, and Topological Tree Optimization in Phylo2Vec Space

Technical Architecture & Evaluation Report

November 2026

Abstract

Balanced Minimum Evolution (BME) is one of the most statistically consistent and widely used distance-based criteria for phylogenetic tree inference and underlies standard tools such as FastME. Classical BME optimizers represent trees as pointer-linked graph structures and perform sequential Subtree Prune and Regraft (SPR) hill-climbing, applying a single topological move per $O(N^2)$ sweep. In contrast, the Phylo2Vec representation bijectively encodes any rooted binary tree on N leaves as an integer vector $v \in \mathbb{Z}^{N-1}$ satisfying $0 \leq v_i \leq 2i$. In this paper, we present a mathematical formulation and a high-performance software implementation for exact $O(N^2)$ BME discrete gradient computation and topological optimization directly in Phylo2Vec vector space. First, we derive the complete $(N-1) \times (2N-3)$ discrete gradient matrix $\nabla_{\text{disc}} L(v)$ over all valid coordinate values $0 \leq j \leq 2i$, paired with an $O(N)$ tree-based top-down cherry-first gauge transformation and an inverse tree-to-vector bijection. Second, we introduce a node-disjoint multi-coordinate (K-SPR) batch optimizer with dual-mode gauge rotation and stochastic 2-SPR basin hopping that applies multiple non-interfering SPR moves per $O(N^2)$ gradient evaluation. Third, to scale to very large trees on multi-core CPUs and GPUs, we prove five architectural theorems: (i) the *Column-Independence Theorem* for lockstep SIMT clade-distance dynamic programming, (ii) the *Stackless Ancestor-Walk Top-Down Theorem* ($O(N \log N)$ expected work on balanced trees, $O(N^2)$ worst-case on caterpillars, with $O(1)$ stack space), (iii) the *Branchless Cherry-First Tree Decoding and Heavy-First* $\lceil \log_2(2N) \rceil$*-Slot Register-Stack Theorem* with 3-edge incident zeroing, $O(1)$ incremental destination mapping, and race-free SIMT parallel destination indexing, (iv) the *Complete Column-Autonomy Theorem* for 4× memory-reduced column tiling paired with fused top-1 row candidate reductions, and (v) the *Exact Clade-Condensation Theorem for Minibatch BME Optimization*, which collapses inactive clades into dyadic-weighted super-leaves to evaluate exact full-tree BME discrete gradients on active subtrees of size $N_{\text{eff}} \ll N$. Across 120 benchmark datasets ($N = 20 \ldots 200$), our optimizer achieves a 99.2% win/tie rate (119/120 trials) in BME tree length against FastME while requiring up to 3.32× fewer $O(N^2)$ evaluations and running up to 4.16× faster on Apple M5 and up to 5.74× faster on AMD Threadripper PRO 5995WX (single core). At large scale ($N = 100 \ldots 10{,}000{,}000$), our column-parallel and GPU-accelerated kernels achieve up to a 9.01× `subdist` speedup (5.98× end-to-end hybrid speedup) on an Apple M5 Metal GPU, up to a 62.5× single-GPU speedup on an NVIDIA RTX A6000 and a 112.8× single-GPU speedup at $N = 4{,}000$ (6.22 ms vs. 701.38 ms) on an NVIDIA H100 80GB HBM3, 5.15× end-to-end (7.70× Stage-3) scaling on 8× NVIDIA H100 at $N = 40{,}000$ (158.36 ms for 3.2×10^9 moves), distributed column-sharded full-tree execution across 8× NVIDIA H100 up to $N = 380{,}000$ taxa (2.89×10^{11} candidate moves in 205.45 s across 569.9 GB HBM3), and clade-condensed minibatch optimization at $N = 1{,}000{,}000$, 5,000,000, and 10,000,000 taxa (3.43 s, 17.82 s, and 39.20 s per full epoch on 8× NVIDIA H100 using 7.8 GB VRAM/GPU). Finally, on 1,596-bp `JC69` and biophysical **RNASim** 16S rRNA sequence benchmarks up to $N = 100{,}000$ taxa, our GPU optimizer outperforms both 64-thread `FastME 2.1.6` and 64-thread `VeryFastTree 4.0.5` in Robinson–Foulds topological accuracy across every tested scale while running up to 4,087× faster than `FastME` and 142 × –688× faster than `VeryFastTree` (8,805 vs. 8,857 wrong splits at $N = 100{,}000$ in 3.46 s vs. 772.53 s).

Contents

1 Introduction **2**
- 1.1 Contributions and Architecture Overview . . . 3

2 Mathematical Formulation of the Phylo2Vec Discrete Gradient **4**
- 2.1 Phylo2Vec Tree Decoding and Unrooted Representation . . . 4
- 2.2 Cherry-First Gauge Transformation and Exact Inverse Bijection . . . 4
- 2.3 Two-Pass $O(N^2)$ Balanced Clade Distance Matrix (`subdist`) . . . 6
- 2.4 Half-Tree SPR Recurrence and the Full $0 \le j \le 2i$ Gradient Mapping . . . 6

3 Topological Optimization via Node-Disjoint K-SPR Search **7**
- 3.1 Node-Disjoint Path Corridors and Batch Line Search . . . 7
- 3.2 4-Stage Phylo2Vec BME Optimization Pipeline . . . 8

4 Parallel Multi-Core CPU, SIMT GPU, and Column-Tiled Architecture **8**
- 4.1 Column-Independence Theorem for Lockstep SIMT Warps . . . 8
- 4.2 Stackless Ancestor-Walk Top-Down Theorem . . . 9
- 4.3 Heavy-First $\lceil \log_2(2N) \rceil$-Slot Register Stack and Race-Free Destination Updates . . . 9
- 4.4 Complete Column-Autonomy Theorem for Low-Memory Column Tiling . . . 10
- 4.5 Exact Clade-Condensation Theorem for Multi-Million-Taxon Minibatch Optimization 11

5 Software Architecture and Module Reference **14**

6 Empirical Evaluation and Scaling Analysis **15**
- 6.1 Mathematical Exactness and Sanitizer Verification (`verify_all.c`) . . . 15
- 6.2 End-to-End Topological Optimization vs. FastME (`bench_optimizers.c`) . . . 16
- 6.3 Multi-Core CPU, Column-Tiled, and Metal GPU Scaling (`bench_parallel_gpu.m`) 17
- 6.4 Empirical Multi-GPU Scaling on 2x NVIDIA RTX A6000 and 8x NVIDIA H100 80GB HBM3 . . . 18

7 Conclusion **21**

1 Introduction

Under the Balanced Minimum Evolution (BME) framework introduced by Pauplin [1] and formalized by Desper and Gascuel [2], the standard BME length $L(T)$ of an unrooted binary phylogenetic tree T with N leaves and symmetric pairwise leaf distance matrix $D \in \mathbb{R}^{N \times N}$ ($D_{xx} = 0$) is defined over unordered leaf pairs as:

$$L(T) = \frac{1}{2} \sum_{x=0}^{N-1} \sum_{\substack{y=0 \\ y \neq x}}^{N-1} D_{xy}\, 2^{-d_T(x,y)} = \sum_{0 \le x < y \le N-1} D_{xy}\, 2^{-d_T(x,y)} = \sum_{x=0}^{N-1} \sum_{\substack{y=0 \\ y \neq x}}^{N-1} D_{xy}\, 2^{-d_{\text{edge}}(x,y)}, \quad (1)$$

where $d_T(x, y)$ denotes the number of internal degree-3 nodes on the unique path between leaf x and leaf y in T, and $d_{\text{edge}}(x, y) = d_T(x, y) + 1$ is the number of edges on that path. Both our standalone discrete gradient routines (`calculate_discrete_gradient_c` and `calculate_discrete_gradient_full_c`) and our topological optimizers (`optimize_phylo2vec_bme_c` and `optimize_fastme_bme_c`) use the standard unordered Pauplin length $L(T)$ in Eq. (1), so every discrete gradient entry directly reports the exact unordered BME length change $\Delta L = L(T') - L(T)$ (one half of the ordered-pair difference $\Delta L_{\text{ord}} = 2\Delta L$). Desper and Gascuel [2] and Lefort et al. [3] (FastME 2.0) showed that all $O(N^2)$ Subtree Prune and Regraft (SPR) moves on T can be evaluated in $O(N^2)$ total time via a two-pass recursive tree traversal that maintains balanced clade-to-clade average distances.

While FastME's $O(N^2)$ per-sweep complexity is asymptotically optimal for evaluating all SPR moves of a fixed tree, pointer-based graph representations impose three computational limitations:

1. **Single-Move Sequential Trajectory ($\Theta(N^3)$ Scaling):** FastME identifies the single best SPR move across all $O(N^2)$ candidates, mutates the pointer graph for that one move, and recomputes the entire $(2N-2) \times (2N-2)$ balanced averages table from scratch on the next iteration. Because optimizing an N-leaf tree requires $\Theta(N)$ SPR moves, total optimization time scales as $\Theta(N^3)$ sequential sweeps.

2. **Irregular Pointer Chasing:** Representing trees with heap-allocated node and edge structs prevents SIMD vectorization and SIMT GPU execution, and causes CPU cache and TLB misses once the clade-distance table exceeds L2/L3 cache ($N \geq 1{,}500$).

3. **Lack of Algebraic Coordinate Gradients:** Pointer graphs do not expose a fixed-dimensional coordinate domain for discrete gradient descent, batch coordinate updates, or variational phylogenetic inference.

Recently, Penn et al. [4] introduced **Phylo2Vec**, a bijective representation that maps any rooted binary phylogenetic tree on N labeled leaves $\{0, 1, \ldots, N-1\}$ to an integer vector $v = (v_0, v_1, \ldots, v_{N-2}) \in \mathcal{V}$, where:

$$\mathcal{V} = \left\{ (v_0, v_1, \ldots, v_{N-2}) \in \mathbb{Z}^{N-1} \;\middle|\; 0 \leq v_i \leq 2i \quad \forall\, i \in \{0, \ldots, N-2\} \right\}, \qquad |\mathcal{V}| = (2N-3)!!. \quad (2)$$

Perturbing a single coordinate $v_i \leftarrow j$ ($j \in \{0, \ldots, 2i\}$) corresponds to a structured SPR move in which the subtree attached at step i is pruned and regrafted onto the edge indexed by j in the prefix tree on leaves $\{0, \ldots, i\}$.

1.1 Contributions and Architecture Overview

In this paper, we describe a mathematical and software framework for BME discrete gradient computation and topological tree optimization in Phylo2Vec space:

1. **Full $0 \leq j \leq 2i$ Discrete Gradient in $O(N^2)$ Time (Section 2):** For any tree in cherry-first canonical gauge ($0 \leq v_i \leq i$), we compute the complete $(N-1) \times (2N-3)$ discrete gradient matrix $\Delta_{i,j} = L(v^{(i \leftarrow j)}) - L(v)$ across all $2i+1$ valid regrafting coordinates $0 \leq j \leq 2i$, which cover leaf-incident pendant edges ($0 \leq j \leq i$), internal clade edges ($i+1 \leq j \leq 2i-2$), and root-incident edges ($j \in \{2i-1, 2i\}$ with the boundary guard $2i-1 > i$ at row $i = 1$).

2. **Cherry-First Gauge Transformation and Exact Inverse Bijection (Section 2.2):** We prove that every decoded binary tree can be relabeled into the cherry-first subset $\mathcal{V}_{\text{cf}} = \{v \in \mathcal{V} : 0 \leq v_i \leq i\}$ in $O(N)$ time via a single top-down pass that issues leaf labels in birth order (`relabel_from_rooted_tree`), and we provide an exact inverse bijection (`encode_rooted_to_v_c` and `encode_unrooted_to_v_c`) that recovers $v \in \mathcal{V}$ from any rooted or unrooted tree topology.

3. **Node-Disjoint Multi-Coordinate (K-SPR) Batch Optimizer (Section 3):** We design a hybrid optimizer that extracts multiple mutually node-disjoint improving coordinate moves from a single discrete gradient evaluation and applies K non-interfering SPR moves at once, with exact line-search verification, dual-mode gauge rotation, and 2-SPR basin hopping.

4. **SIMT GPU, Multi-Core CPU, Column-Tiled, and Clade-Condensed Minibatch Algorithms (Section 4):** We prove five theorems that transform the tree-based recurrences into lockstep vector/matrix operations and enable multi-million-taxon scaling: the *Column-Independence Theorem*, the *Stackless Ancestor-Walk Theorem*, the *Heavy-First* $\lceil \log_2(2N) \rceil$*-Slot Register-Stack Theorem* with $O(1)$ sequential and race-free SIMT destination mapping, the *Complete Column-Autonomy Theorem* for 4× memory-reduced column tiling, and the *Exact Clade-Condensation Theorem for Minibatch BME Optimization.*

2 Mathematical Formulation of the Phylo2Vec Discrete Gradient

2.1 Phylo2Vec Tree Decoding and Unrooted Representation

Given a vector $v \in \mathcal{V}_{\text{cf}}$ in the cherry-first canonical domain ($0 \le v_i \le i$ for all $i \in \{0, \dots, N-2\}$), the corresponding rooted binary tree T_{root} with $2N-1$ nodes (leaves $0 \dots N-1$ and internal nodes $N \dots 2N-2$) is decoded in $O(N)$ time in reverse coordinate order $k = 0, \dots, N-2$ (where $i = N-2-k$):

$$\text{pairs}[k] = (p_1, p_2) = (v_i,\, i+1), \qquad m = N + k. \tag{3}$$

Using a path-compressed union-find array parents_uf$[0 \dots 2N-2]$ initialized to the identity, each step k finds the current subtree roots $r_1 = \text{find}(p_1)$ and $r_2 = \text{find}(p_2)$, creates internal node $m = N + k$ with children $c_1(m) = r_2$ and $c_2(m) = r_1$, and sets parents_uf$[r_1]$ = parents_uf$[r_2] = m$. (For general vectors $v \in \mathcal{V}$ with $v_i > i$, forward array-insertion decoding constructs T_{root} in $O(N^2)$ worst-case time.)

Because the BME tree length $L(T)$ in Eq. (1) is invariant to the placement of the root on the top edge, we collapse the root node $2N-2$ of the rooted tree (whose children are $R = 2N-3$ and $c_{\text{other}} = \min(c_1(2N-2), c_2(2N-2))$) into an unrooted tree T on $M = 2N-2$ nodes $\{0, 1, \dots, 2N-3\}$ rooted at the degree-3 trifurcating node $R = 2N-3$ with three children:

$$R_{c_0} = c_1(R), \qquad R_{c_1} = c_2(R), \qquad R_{c_2} = c_{\text{other}}. \tag{4}$$

Every internal node $u \in \{N, \dots, R-1\}$ has two downward children $c_1(u), c_2(u)$ and one upward parent parent_u$[u]$, forming a 3-regular adjacency table adj_u$[u, 0..2] = (c_1(u), c_2(u), \text{parent_u}[u])$, while adj_u$[R, 0..2] = (R_{c_0}, R_{c_1}, R_{c_2})$. We compute subtree node counts sub_size$[u]$ bottom-up and assign contiguous depth-first search (DFS) entry times $\text{tin}[u] \in \{0, \dots, 2N-3\}$ top-down:

$$\begin{aligned}
\text{tin}[R] = 0, \quad \text{tin}[R_{c_0}] = 1, \quad \text{tin}[R_{c_1}] &= 1 + \text{sub_size}[R_{c_0}], \\
\text{tin}[R_{c_2}] &= \text{tin}[R_{c_1}] + \text{sub_size}[R_{c_1}], \\
\text{tin}[c_1(u)] = \text{tin}[u] + 1, \quad \text{tin}[c_2(u)] &= \text{tin}[u] + 1 + \text{sub_size}[c_1(u)] \qquad (u = R-1 \text{ down to } N).
\end{aligned} \tag{5}$$

By construction, a node v lies in the strict downward subtree $T_u \setminus \{u\}$ of internal node u if and only if $\text{tin}[u] < \text{tin}[v] < \text{tin}[u] + \text{sub_size}[u]$, and for $v \in T_u \setminus \{u\}$, v lies in $T_{c_1(u)}$ if and only if $\text{tin}[v] < \text{tin}[c_2(u)]$.

2.2 Cherry-First Gauge Transformation and Exact Inverse Bijection

While general Phylo2Vec vectors satisfy $0 \le v_i \le 2i$, the $O(N)$ bottom-up union-find decoding above requires the *cherry-first canonical gauge* $0 \le v_i \le i$ (in which every merge occurs on a leaf pendant edge rather than an internal ancestral edge). We establish that any already-decoded rooted binary tree can be transformed into cherry-first gauge in $O(N)$ time via a leaf permutation $\pi \in S_N$ (`relabel_from_rooted_tree`), and conversely any rooted or unrooted tree can be encoded back into $v \in \mathcal{V}$ ($0 \le v_i \le 2i$).

Theorem 2.1 (Tree-Based $O(N)$ Cherry-First Gauge Transformation and Clade-Flip Duality)**.** *Let T be any rooted binary tree on N leaves with child arrays c_1, c_2 and root $2N-2$, whose internal nodes $u \in \{N, \dots, 2N-2\}$ are indexed so that every parent has a larger index than its children (as produced by Phylo2Vec decoding). For every node $u \in \{0, \dots, 2N-2\}$, define the minimum leaf in u's clade bottom-up by $m(x) = x$ for $x < N$ and $m(u) = \min(m(c_1(u)), m(c_2(u)))$ for $u \ge N$. Fix a clade orientation flip_mode $\in \{0, 1\}$, set $\ell(2N-2) = 0$ and a label counter $t = 1$, and visit the internal nodes top-down in decreasing index order $u = 2N-2, 2N-3, \dots, N$:*

- *If flip_mode $= 0$, order the children (a, b) of u so that $m(a) < m(b)$; if flip_mode $= 1$, order (a, b) so that $m(a) > m(b)$.*

- *Set $\ell(a) = \ell(u)$ and $\ell(b) = t$, record the cherry-first coordinate $v^{cf}_{t-1} = \ell(a)$, and increment t.*

Then $\pi(k) = \ell^{-1}(k)$ defines a bijection on $\{0, \dots, N-1\}$ computed in $O(N)$ time from T such that $v^{cf} \in \mathcal{V}_{cf}$ ($0 \le v^{cf}_i \le i$ for all i) encodes the exact same tree T under the permuted distance matrix $D_\pi[a,b] = D[\pi(a), \pi(b)]$, with $L(v^{cf}; D_\pi) = L(v; D)$. If T was decoded from some $v \in \mathcal{V}_{cf}$, then flip_mode $= 0$ returns $v^{cf} = v$ and $\pi = id$. (When starting from an undecoded general vector $v \in \mathcal{V}$ with $v_i > i$, initial array-insertion decoding into T and explicit materialization of the $N \times N$ matrix D_π take $O(N^2)$ time.)

Proof. Write $t(u) = \ell(b)$ for the label issued at internal node u. The counter issues each of $1, \dots, N-1$ exactly once. Every label, including the root label 0, is passed unchanged from a node to its a-child and so travels down a chain of a-children to exactly one leaf; since each node receives exactly one label from its parent, ℓ restricted to the leaves is a bijection onto $\{0, \dots, N-1\}$. The label $\ell(a) = \ell(u)$ was issued at a proper ancestor of u (or is the root label 0), hence before $t(u)$, so $v^{\mathrm{cf}}_{t(u)-1} = \ell(u) < t(u)$ and $v^{\mathrm{cf}} \in \mathcal{V}_{\mathrm{cf}}$. Because parents are visited before their children, the issue order is heap-ordered: $t(w) > t(u)$ for every internal descendant w of u.

Decoding v^{cf} processes $i = N-2$ down to 0, merging the clades that contain leaves v^{cf}_i and $i+1$. We show by induction that step i creates exactly the clade T_u of the node with $t(u) = i+1$. Every internal node of T_a and T_b is a descendant of u, so by heap order it has $t > i+1$ and its merge was completed at an earlier step, whereas u itself has not yet been processed; thus T_a and T_b are both complete, separate components. Leaf $\ell(b) = i+1$ lies in T_b and leaf $\ell(a) = v^{\mathrm{cf}}_i$ lies in T_a, so step i merges precisely T_a and T_b, producing T_u. After step 0 every clade of T has been produced, so v^{cf} encodes T under the labels ℓ, i.e., under D_π, and the BME length is unchanged. The argument never uses which child is a, so it holds for both values of flip_mode. Finally, if T was decoded from $v \in \mathcal{V}_{\mathrm{cf}}$, the node created at step i has index $N + (N-2-i)$ and children with minimum leaves v_i and $i+1$ (all other leaves in these two components exceed $i+1$); it is therefore the $(i+1)$-th node visited, flip_mode $= 0$ issues $t = i+1$ to the child containing leaf $i+1$, and induction on i gives $\ell = \mathrm{id}$ and $v^{\mathrm{cf}} = v$. □

Intuitively, v^{cf} is a birth history of T: reading the coordinates forward, leaf $i+1$ is born at time $i+1$ from the existing lineage v^{cf}_i, the internal-node index order supplies the ranking of birth events, and at every split the a-child continues its parent's lineage label while the b-child receives the next label. The labels must follow a top-down ranking: assigning T_b a contiguous depth-first interval starting at $\ell(u) + |T_a|$ violates heap order whenever T_a contains an internal node (e.g., for $((x_0, x_1), (x_2, x_3))$ it yields $(0, 0, 2)$, which decodes to a caterpillar).

Theorem 2.2 (Exact Inverse Bijection from Rooted and Unrooted Trees to $\mathcal{V}$)**.** *Given any rooted binary tree T on leaves $\{0, \dots, N-1\}$ (or any unrooted binary tree T canonically rooted on the pendant edge $(0, adj_u[0,0])$ incident to leaf 0 via post-order traversal in `encode_unrooted_to_v_c`), let each internal node $m \in \{N, \dots, 2N-2\}$ combine child clades $c_1(m), c_2(m)$ with minimum leaves $l_{\min}(m) = \min(m(c_1(m)), m(c_2(m)))$ and $l_{\max}(m) = \max(m(c_1(m)), m(c_2(m)))$. Then $l_{\max}(m)$ takes each value in $\{1, \dots, N-1\}$ over the $N-1$ internal nodes. Processing step $i = 0, \dots, N-2$ (for leaf $i+1$) along the ancestor chain from leaf $i+1$'s parent $p = parent[i+1]$ up to the current active root in the prefix tree on $\{0, \dots, i\}$ (`encode_rooted_to_v_c`):*

- *If p is the first active ancestor encountered on the lineage of sibling minimum leaf $s = m(sib(i+1)) < i+1$, then leaf $i+1$ attaches directly to the pendant edge of leaf s, so $v_i = s \in \{0, \dots, i\}$.*
- *Otherwise, walking down the active ancestor chain of leaf s to locate the predecessor internal node q whose parent in the prefix tree is p yields the internal edge label $v_i = edge_label[q] \in \{i+1, \dots, 2i\}$, after which p is activated with $edge_label[p] = 2i+1$.*

This recovers the unique vector $v \in \mathcal{V}$ in $O(N)$ time on cherry-first trees and $O(N \cdot height(T))$ time ($O(N \log N)$ expected, $O(N^2)$ worst-case) on general rooted trees.

2.3 Two-Pass $O(N^2)$ Balanced Clade Distance Matrix (`subdist`)

To evaluate all SPR moves in $O(N^2)$ time, we construct the symmetric $(2N-2)\times(2N-2)$ balanced clade distance matrix $\mathrm{subdist}[u,v]$, which stores the directional balanced average distance between the subtrees induced by every pair of nodes (u,v) in the unrooted tree T:

1. **Leaf Initialization ($0 \le x, y < N$):** $\mathrm{subdist}[x,y] = D_{xy}$.

2. **Bottom-Up Clade Recurrence ($u = N, \ldots, R-1$):** For any node $v \notin T_u$, the balanced average distance from u's downward subtree T_u to v satisfies:

$$\mathrm{subdist}[u,v] = \mathrm{subdist}[v,u] = \frac{1}{2}\left(\mathrm{subdist}[c_1(u),v] + \mathrm{subdist}[c_2(u),v]\right). \tag{6}$$

3. **Trifurcating Root 3-Way Recurrence ($u = R = 2N-3$):** For each child R_{c_k} ($k \in \{0,1,2\}$) with complementary root children $R_{c_{k'}}, R_{c_{k''}}$ ($k', k'' \neq k$), every node $v \in T_{R_{c_k}}$ has:

$$\mathrm{subdist}[R,v] = \mathrm{subdist}[v,R] = \frac{1}{2}\left(\mathrm{subdist}[R_{c_{k'}},v] + \mathrm{subdist}[R_{c_{k''}},v]\right). \tag{7}$$

4. **Top-Down Complementary Clade Recurrence ($p = R-1$ down to N):** When $v \in T_p \setminus \{p\}$ lies inside the downward subtree of p, the clade at p oriented *away* from v consists of p's parent $g = \mathrm{parent_u}[p]$ and the sibling child $\mathrm{sib}_p(v) \in \{c_1(p), c_2(p)\}$ not containing v:

$$\mathrm{subdist}[p,v] = \mathrm{subdist}[v,p] = \frac{1}{2}\left(\mathrm{subdist}[\mathrm{sib}_p(v),v] + \mathrm{subdist}[\mathrm{parent_u}[p],v]\right). \tag{8}$$

5. **Diagonal and Local Neighbor Distances (`nd_u`):** Setting $\mathrm{subdist}[u,u] = 0$ for all u, we cache for each internal node $u \in \{N, \ldots, R\}$ with neighbors $(n_0, n_1, n_2) = \mathrm{adj_u}[u, 0..2]$ the three pairwise neighbor-clade distances:

$$\mathrm{nd_u}[u,0] = \mathrm{subdist}[n_1,n_2], \quad \mathrm{nd_u}[u,1] = \mathrm{subdist}[n_0,n_2], \quad \mathrm{nd_u}[u,2] = \mathrm{subdist}[n_0,n_1]. \tag{9}$$

2.4 Half-Tree SPR Recurrence and the Full $0 \le j \le 2i$ Gradient Mapping

For each row $i \in \{1, \ldots, N-2\}$ (corresponding to leaf index $v_{\mathrm{index}} = i+1$), the subtree pruned when coordinate v_i changes is $T_{\mathrm{prune}} = \mathrm{v_map}[v_{\mathrm{index}}]$, which was attached to internal node $P = \mathrm{parent_u}[T_{\mathrm{prune}}]$ (or c_{other} if $T_{\mathrm{prune}} = R$). Removing T_{prune} and contracting P leaves an edge between P's other two neighbors, other_0 and other_1, partitioning the remaining tree into two disjoint half-trees rooted at other_0 and other_1.

Traversing into the half-tree starting at $u_0 = \mathrm{other}_0$ (with opposite anchor $\mathrm{orig} = \mathrm{other}_1$ and initial distance $d_{kC}^{(0)} = \mathrm{subdist}[T_{\mathrm{prune}}, \mathrm{orig}]$), each step across an internal node u from predecessor C branches into the two outgoing neighbors b_0, b_1 of u with path-length weight $w_\ell = 2^{-(\ell+1)}$. Updating the path-adjusted cross-clade distance D_{AC} and pruned-clade distance d_{kC}:

$$\begin{aligned}
D_{AC}(b_0) &= \mathrm{subdist}[C,b_1]^{\mathrm{before}} + \left(\mathrm{subdist}[\mathrm{orig},b_1] - \mathrm{subdist}[T_{\mathrm{prune}},b_1]\right) 2^{-(\ell+1)}, \\
\Delta L(u \to b_0) &= \Delta L(C \to u) + \frac{1}{4}\left(D_{AC}(b_0) + \mathrm{subdist}[T_{\mathrm{prune}},b_0] - D_{AB}(u) - d_{kC}(u)\right), \\
d_{kC}(b_0) &= \frac{1}{2}\left(d_{kC}(u) + \mathrm{subdist}[T_{\mathrm{prune}},b_1]\right),
\end{aligned} \tag{10}$$

and symmetrically for b_1, we store $\Delta L(u \to b)$ in $\mathrm{edge_delta}[c_{\mathrm{end}}(u,b)]$, where $c_{\mathrm{end}}(u,b) \in \{0, \ldots, 2N-3\}$ is the child endpoint of the edge (u,b) in the unrooted tree rooted at R. Running this recurrence on both $(\mathrm{other}_0, \mathrm{other}_1)$ and $(\mathrm{other}_1, \mathrm{other}_0)$ populates $\mathrm{edge_delta}[0 \ldots 2N-3]$ in $O(N)$ time per row i.

Theorem 2.3 (Complete $0 \le j \le 2i$ Edge-to-Coordinate Mapping)**.** *For any cherry-first vector $v \in \mathcal{V}_{cf}$ ($v_i \le i$) and row $i \in \{1, \dots, N-2\}$, the change in Pauplin BME tree length $\Delta_{i,j} = L(v^{(i \leftarrow j)}) - L(v)$ for all $2i+1$ valid Phylo2Vec coordinates $j \in \{0, \dots, 2i\}$ is given by:*

$$\Delta_{i,j} = \begin{cases} \mathit{edge_delta}[\mathit{dest_row}_i[j]], & 0 \le j \le i, \\ \mathit{edge_delta}[(2N-2) - 2i + j], & i+1 \le j \le 2i-2, \\ \mathit{edge_delta}[c_{\mathit{other}}], & j \in \{2i-1,\, 2i\} \cap [i+1,\, 2i], \end{cases} \tag{11}$$

where $\mathit{dest_row}_i[j]$ is the highest ancestor of leaf j formed strictly after step i in the bottom-up decoding order ($m \in \{N, \dots, 2N-3-i\}$), and $c_{\mathit{other}} = \min(c_1(2N-2), c_2(2N-2))$ is the non-R child of the rooted root $2N-2$ representing the collapsed top edge of the unrooted tree. In particular, at row $i = 1$, $2i - 1 = 1 = i$ belongs to the leaf-pendant range $[0, i]$, so $j = 1$ receives $\mathit{edge_delta}[\mathit{dest_row}_1[1]]$ (guarded by `if (2 * i - 1 > i)`) while $j = 2i = 2$ receives $\mathit{edge_delta}[c_{\mathit{other}}]$.

Proof. At step i of the Phylo2Vec decoding process, $N-2-i$ internal merges have already occurred (corresponding to internal nodes $m \in \{N, \dots, 2N-3-i\}$). For $0 \le j \le i$, regrafting onto coordinate j attaches T_{prune} to the current root of the clade containing leaf j, which is $\text{dest_row}_i[j]$. For $i+1 \le j \le 2i$, Phylo2Vec indexes internal clades in reverse order of their creation time: $m(i,j) = N + (2i - j)$. Because the unrooted tree collapses the top edge $(2N-2)$ into (R, c_{other}) and shifts internal node indices above the pruned merge by $N-2-i$, the $i-2$ non-root internal edges map bijectively to $u = (2N-2) - 2i + j$ for $i+1 \le j \le 2i-2$, while the rooted edges incident to the root of the prefix tree that satisfy $j > i$ ($j \in \{2i-1, 2i\}$ for $i \ge 2$, and $j = 2$ for $i = 1$) correspond to the unrooted edge indexed by c_{other}. □

3 Topological Optimization via Node-Disjoint K-SPR Search

Evaluating the full $(N-1) \times (2N-3)$ discrete gradient matrix $\Delta = \nabla_{\text{disc}} L(v)$ reveals the exact BME change of all $(N-1)^2$ single-coordinate SPR moves in a single $O(N^2)$ sweep. We use the discrete gradient matrix to apply $K \ge 1$ non-interfering SPR moves simultaneously, instead of a single coordinate update per sweep.

3.1 Node-Disjoint Path Corridors and Batch Line Search

Each candidate coordinate update $v_i \leftarrow j^*$ ($j^* = \arg\min_{0 \le j \le 2i} \Delta_{i,j}$ with $\Delta_{i,j^*} < -\epsilon$) corresponds to moving the pruned subtree $T_{\text{prune}}(i) = \text{v_map}[i+1]$ from its current attachment node $P(i) = \text{parent_u}[T_{\text{prune}}(i)]$ to target edge endpoint $U(i, j^*)$ along a unique path corridor $\mathcal{P}_T(P(i), U(i, j^*))$ in the unrooted tree T.

In the BME length formula (Eq. (1)), an SPR move along $\mathcal{P}_T(P(i), U(i, j^*))$ alters the topological path weights $2^{-d_T(x,y)}$ primarily for paths passing through $\mathcal{P}_T(P(i), U(i, j^*))$. Consequently, if a set of improving coordinate moves $\mathcal{M} = \{(i_1, j_1^*), \dots, (i_K, j_K^*)\}$ satisfies:

1. **Vertex-Disjointness of Corridors:** $\mathcal{P}_T(P(i_a), U(i_a, j_a^*)) \cap \mathcal{P}_T(P(i_b), U(i_b, j_b^*)) = \emptyset$ for all $a \ne b$ (including source and target neighbors), and
2. **Coordinate Prefix Independence:** No move i_a modifies a subtree minimum-leaf index referenced by another move i_b,

then the K moves are topologically non-interfering and their combined BME improvement satisfies $\Delta L(\mathcal{M}) \approx \sum_{k=1}^{K} \Delta_{i_k, j_k^*} < 0$.

Our greedy batch selector (`optimize_phylo2vec_bme_c`) sorts all improving rows i in ascending order of Δ_{i,j^*} (most negative first), marks the tree nodes on $\mathcal{P}_T(P(i), U(i, j^*))$ in a boolean mask blocked$[0 \dots 2N-3]$ as each move is accepted into $\mathcal{M}$, and then performs a backtracking

line search over $K' \in \{|\mathcal{M}|, \lfloor|\mathcal{M}|/2\rfloor, \ldots, 1\}$ using $O(N)$ cherry-first gauge canonicalization and exact $O(N^2)$ BME evaluation. This guarantees strict monotonic descent $L(v^{(t+1)}) < L(v^{(t)})$ on every step, with an exact single-move fallback when $K' = 1$.

3.2 4-Stage Phylo2Vec BME Optimization Pipeline

The topological optimizer (`optimize_phylo2vec_bme_c`) has four stages:

1. **Stage A (Full $0 \le j \le 2i$ Discrete Gradient + Node-Disjoint K-SPR Batch Descent):** Computes $\nabla_{\text{disc}} L(v)$ via Theorem 2.3 and applies node-disjoint multi-coordinate updates until $\min_{i,j} \Delta_{i,j} \ge -10^{-11}$. Whenever a selected coordinate has $j^* > i$, Theorem 2.1 re-canonicalizes $v^{(i \leftarrow j^*)}$ back to the cherry-first domain $v_i^{\text{cf}} \le i$.

2. **Stage B (Dual-Mode $O(N)$ Cherry-First Gauge Rotation):** When a local minimum is reached in the current leaf labeling, the optimizer applies Theorem 2.1 with alternating clade-orientation parity (flip_mode $= 1$ vs. flip_mode $= 0$). Reversing the left-right clade assignment swaps which child subtree acts as the movable leaf $i+1$ and which as the anchor clade at every internal node. This exposes a complementary set of Phylo2Vec coordinate moves and leaves the tree topology unchanged.

3. **Stage C (Remaining Unrooted SPR Scan + Exact Inverse Bijection):** Single-coordinate moves in the two gauges of Stage B reach only part of the unrooted SPR neighborhood. When we enumerate all $2(N-3)(2N-7)$ distinct SPR neighbors by brute force (`verify_spr_coverage.py`, 30 random trees per N), each gauge reaches 27–33% of them and the two gauges together 52–56% for $N = 8 \ldots 20$, and no tested tree was fully covered. Stage C therefore scans the remaining moves directly on the unrooted tree: it prunes every clade that is not indexed by a coordinate v_i ($i \ge 1$), together with the complementary clade above every internal node, and evaluates all of their regraft edges with the same `subdist` table and half-tree recurrence (Eq. (10)) in $O(N^2)$ time. The best improving move is applied, the tree is encoded back into $v \in \mathcal{V}$ via Theorem 2.2 (`encode_unrooted_to_v_c`), and Stage A resumes.

4. **Stage D (Stochastic 2-SPR Basin-Hopping Kicks):** On noisy distance matrices ($\sigma > 0$), single-SPR local minima can trap greedy hill-climbers. After convergence, the optimizer applies a random 2-SPR coordinate perturbation and repeats the descent from the perturbed state; it keeps the best topology found.

4 Parallel Multi-Core CPU, SIMT GPU, and Column-Tiled Architecture

At large N ($N = 1{,}000 \ldots 400{,}000$), naive parallelization of tree dynamic programming fails because of strided column scatter writes, per-level synchronization barriers, and $O(N)$ per-thread BFS queue memory. We prove four theorems that reformulate the Phylo2Vec discrete gradient into lockstep, cache-resident vector and matrix operations, and implement them in `bench_parallel_gpu.m` and `phylo2vec_cuda_8xh100.cu`.

4.1 Column-Independence Theorem for Lockstep SIMT Warps

Theorem 4.1 (Column-Independence of `subdist`). *In the bottom-up recurrence $subdist[u, x] = \frac{1}{2}(subdist[c_1(u), x] + subdist[c_2(u), x])$ ($u = N, \ldots, R-1$), every entry in column $x \in \{0, \ldots, R-1\}$ depends only on earlier rows $c_1(u), c_2(u) < u$ in the same column x.*

Theorem 4.1 has two architectural consequences for SIMT GPUs (Apple Metal and NVIDIA CUDA) and multi-core CPUs:

1. **Zero Per-Level Synchronization Barriers and 100% Coalesced SIMT Warps:** Instead of parallelizing across rows u at the same tree depth (which requires launching a separate GPU kernel at every level of the tree and reading non-contiguous columns), we assign thread gid $= x$ to column x. All 32 threads in a GPU warp iterate over $u = N, \ldots, R-1$ in lockstep (`k_bottom_up_leaf_cols` and `k_bottom_up_int_cols`), issuing 100% coalesced 128-byte cache-line reads and writes (`subdist[u * n_nodes_u + x]`) with zero branch divergence.

2. **Elimination of Strided Scatter Writes via Shared-Memory Tiled Transpose:** Neither the bottom-up pass nor the top-down pass performs strided column scatter writes (`subdist[v * n_nodes_u + u]`) inside the inner loop; instead, a 16×16 shared-memory tiled transpose (`k_transpose_leaf_block` / `gpu_transpose_leaf_block`) copies the rectangular leaf block $[N \ldots R-1, 0 \ldots N-1]$ into $[0 \ldots N-1, N \ldots R-1]$ between Pass 1 and Pass 2 with fully coalesced global memory reads and writes.

4.2 Stackless Ancestor-Walk Top-Down Theorem

Theorem 4.2 (Stackless Ancestor-Walk Top-Down Pass)**.** *In the top-down recurrence (Eq. (8)), entry subdist$[p, v]$ is updated if and only if $v \in T_p \setminus \{p\}$, i.e., if and only if p is a strict ancestor of v in the unrooted tree rooted at R. Moreover, given the Euler-tour entry times tin in Eq. (5), thread $v \in \{0, \ldots, R-1\}$ can traverse the exact top-down ancestor path from R to v in depth(v) steps with $O(1)$ register memory and zero stack space:*

$$next_curr = \begin{cases} c_1(p), & \text{if } tin[v] < tin[c_2(p)], \\ c_2(p), & \text{otherwise}, \end{cases} \qquad sib = \begin{cases} c_2(p), & \text{if } tin[v] < tin[c_2(p)], \\ c_1(p), & \text{otherwise}, \end{cases} \tag{12}$$

writing both subdist$[p, v]$ and the symmetric entry subdist$[v, p]$ in-place without data races across threads v. Across all $2N-2$ threads, total work is $\sum_{v=0}^{2N-3} depth(v)$, which is $O(N \log N)$ in expectation on balanced or Yule–Harding random trees ($\max_v depth(v) = O(\log N)$) and $O(N^2)$ in the worst case on degenerate caterpillar trees ($\max_v depth(v) = O(N)$).

Proof. Because $T_{c_1(p)}$ occupies $[\text{tin}[p] + 1, \text{tin}[c_2(p)))$ and $T_{c_2(p)}$ occupies $[\text{tin}[c_2(p)], \text{tin}[p] + \text{sub_size}[p])$, the single comparison $\text{tin}[v] < \text{tin}[c_2(p)]$ determines which child of p contains v. At ancestor p of v, thread v reads subdist$[\text{sib}, v]$ (where $T_{\text{sib}} \cap T_p^{\text{child}} = \emptyset$, so subdist$[\text{sib}, v]$ was already finalized in the bottom-up pass) and subdist$[\text{parent_u}[p], v]$ (computed by thread v on the immediately preceding step of its top-down walk). Conversely, the symmetric write subdist$[v, p]$ modifies column p at row $v \in T_p \setminus \{p\}$, which is never read by thread p because thread p only reads rows outside T_p. Thus all threads $v \in \{0, \ldots, R-1\}$ execute concurrently with zero race conditions. □

4.3 Heavy-First $\lceil \log_2(2N) \rceil$-Slot Register Stack and Race-Free Destination Updates

Theorem 4.3 (Branchless Cherry-First Tree Decoding, Logarithmic Stack Bound, and Race-Free Destination Mapping)**.** *The tree setup and Stage 3 SPR sweep satisfy the following three structural invariants:*

1. ***Branchless $O(N)$ Cherry-First Component-Root Invariant:*** *For any cherry-first vector $v \in \mathcal{V}_N^{cf}$ ($0 \le v_i \le i$), at merge step $k = 0, \ldots, N-2$ ($i = N-2-k$), the active forest partitions $\{0, \ldots, N-1\}$ into $i+2$ connected components $C_0, \ldots, C_{i+1}$ whose minimum leaves are $\min(C_x) = x$. Consequently, initializing root$[x] \leftarrow x$ for $x \in \{0, \ldots, N-1\}$ allows the rooted tree topology $(c_1(m), c_2(m))$, minimum-leaf table $l_{\min}(m) = v_i$, and pruned-subtree lookup v_map$[i+1] =$ root$[i+1]$ to be constructed in a single branchless $O(N)$ loop (child$_1[N+k] \leftarrow$ root$[i+1]$, child$_2[N+k] \leftarrow$ root$[v_i]$, root$[v_i] \leftarrow N+k$), with no intermediate `pairs` array and no union-find path compression.*

2. ***Logarithmic Stack Bound and 3-Edge Incident Zeroing:*** *In the Stage 3 half-tree traversal (`run_half_tree_dfs`), each internal node u visited from C branches into two neighbors b_0, b_1 with remaining component sizes $S(u \to b_k) = sub_size[b_k]$ if $parent_u[b_k] = u$ and $(2N-2) - sub_size[u]$ otherwise. Pushing the neighbor with the larger component size onto a LIFO stack first and the smaller neighbor second guarantees that the stack depth never exceeds $\lfloor \log_2(2N-2) \rfloor + 2$ for any binary tree topology (at most 16 slots for $N \leq 32{,}768$ and at most 20 slots for $N \leq 500{,}000$). Moreover, because every coordinate $0 \leq j \leq 2i$ maps to an edge outside T_{tree_root} and `run_half_tree_dfs` overwrites every non-incident edge outside T_{tree_root}, only the three edges incident to parent (with child endpoints $e(w) = w$ if $parent_u[w] = parent$ else $parent$, for $w \in \{tree_root, other_0, other_1\}$) need to be zero-initialized before traversal. This removes $O(N^2)$ redundant memory stores across all $N-2$ rows and prevents inter-warp write conflicts with $other_0, other_1$ half-tree traversals.*

3. $O(1)$ ***Sequential and Race-Free Parallel Destination Mapping:*** *In bottom-up Phylo2Vec merge order $m = N + k$ ($k = 0, \ldots, N-3$), the larger minimum leaf of merge m is $l_{\max}(N+k) = N-1-k$. Consequently, as row index $i = v_{index} - 1$ steps downward from $N-2$ to 1 on a CPU, the destination lookup array $dest_row[0 \ldots N-1]$ (initialized to $dest_row[j] = j$ at $i = N-2$) changes in only one entry per row:*

$$dest_row[l_{\min}(N+k)] \leftarrow N+k \qquad \text{for } k = N-2-i, \tag{13}$$

so the $O(N^2)$ destinations matrix is not needed. For independent SIMT GPU threadblocks executing row i with $n_{active} = N-2-i$ active merges, precomputing on the host the successor index $next_k_same_lmin[k] = \min\{k' > k : l_{\min}(N+k') = l_{\min}(N+k)\}$ (or N if none exists) ensures that for each leaf column j, at most one active merge $k \in \{0, \ldots, n_{active}-1\}$ satisfies $next_k_same_lmin[k] \geq n_{active}$, so all threads k in a threadblock can populate $\Delta_{i, l_{\min}(N+k)} = edge_delta[N+k]$ concurrently without write-after-write races.

4.4 Complete Column-Autonomy Theorem for Low-Memory Column Tiling

Theorem 4.4 (Complete Column-Autonomy and 4× `subdist` Memory Reduction via Column Tiling)**.** *Suppose the bottom-up recurrence (Eq. (6)) and the stackless ancestor-walk top-down pass (Theorem 4.2) are executed on the N leaf columns $x \in \{0, \ldots, N-1\}$ in tiles of width B_{tile} (e.g., $B_{tile} = 128$), storing only the internal-to-leaf table $subdist_int_leaf[w-N, x] = subdist[w, x]$ of dimension $(N-1) \times N$ ($\frac{1}{4}$ the size of subdist). Then for any node $w \in \{0, \ldots, 2N-3\}$, its entire row $row_w[0 \ldots 2N-3] = subdist[w, \cdot]$ can be reconstructed autonomously in $O(N + depth(w)) = O(N)$ time inside a single $2N$-element L1/shared-memory buffer (`compute_autonomous_row_f64` / `dev_compute_autonomous_row`):*

1. *Copy $row_w[0 \ldots N-1] \leftarrow D[w, 0 \ldots N-1]$ (if $w < N$) or $subdist_int_leaf[w-N, 0 \ldots N-1]$ (if $w \geq N$).*
2. *For $u = N, \ldots, R-1$: set $row_w[u] = \frac{1}{2}(row_w[c_1(u)] + row_w[c_2(u)])$.*
3. *Walk top-down along the depth(w) ancestors p from R to w, setting $row_w[p] = \frac{1}{2}(row_w[sib_p(w)] + row_w[parent_u[p]])$, and finally set $row_w[w] = 0$.*

Proof. By symmetry of subdist, $\text{row}_w[x] = \text{subdist}[w, x] = \text{subdist}[x, w]$ for all leaves $x \in \{0, \ldots, N-1\}$. For any internal node $u \in \{N, \ldots, R-1\}$: (i) if $T_u \cap T_w = \emptyset$, the bottom-up step averages $\text{subdist}[w, x]$ over $x \in T_u$, yielding $\text{subdist}[w, u]$; (ii) if $u \in T_w \setminus \{w\}$ is a strict descendant of w, then T_u lies entirely within a single child subtree of w, where every leaf $x \in T_u$ already holds its exact top-down distance $\text{subdist}[w, x] = \frac{1}{2}(\text{subdist}[\text{sib}_w(u), x] + \text{subdist}[\text{parent_u}[w], x])$; by linearity of balanced subtree averaging over $x \in T_u$, the bottom-up pass on row_w yields the exact top-down value $\text{row}_w[u] = \text{subdist}[w, u]$ for all $u \in T_w \setminus \{w\}$; and (iii) if u is a strict ancestor of w ($w \in T_u \setminus \{u\}$), Step 3 overwrites $\text{row}_w[u]$ using $\text{row}_w[\text{sib}_u(w)]$ (which falls into Case (i) since $T_{\text{sib}_u(w)} \cap T_w = \emptyset$) and $\text{row}_w[\text{parent_u}[u]]$. □

Because Stage 3 for row v_{index} accesses only three rows of subdist ($\text{row}_{\text{tree_root}}$, $\text{row}_{\text{other}_0}$, and $\text{row}_{\text{other}_1}$), Theorem 4.4 allows each CPU thread or GPU threadblock to reconstruct those three rows directly inside private L1/L2 cache or GPU shared memory in $O(N)$ time per row ($O(N^2)$ total across all $N-1$ rows) without ever allocating the $(2N-2) \times (2N-2)$ subdist matrix. On SIMT GPUs (`phylo2vec_cuda_8xh100.cu`), we further accelerate Stage 3 and column-tiled reconstruction with two mechanisms designed around the GPU hardware:

1. **Heavy-Threshold Parallel Subtree Seeding with Register-Continued Heavy-First DFS (`dev_run_parallel_spr_dfs`):** Expanding the half-tree level by level across all 128 threads, with a shared-memory barrier at every depth, incurs $O(N)$ barriers on skewed trees and leaves threads idle near the root. Instead, a compact FIFO seeding phase (`dev_seed_half_tree_fifo`) splits only internal clades larger than a heavy size threshold $\tau_{\text{heavy}} = \max(8, \min(64, (2N-2)/96))$ into a shared-memory queue (`SprSharedQueue`, 5,136 B) of at most 128 subtree roots, and immediately evaluates leaf and 2-leaf cherry clades in place (`dev_eval_cherry_or_enqueue`). Each thread then drains a subtree root and traverses its entire remaining subtree independently in registers using an 8-slot Heavy-First LIFO stack (`dev_expand_subtree_stack`) with zero `__syncthreads()` barriers.

2. **Lockstep Batch-Tiled Column Reconstruction (`k_load_auto_leaves_tiled`, `k_reconstruct_auto_cols`, `k_transpose_auto_cols`):** For each batch of B_{row} SPR rows in the column-tiled pipeline (`k_spr_rows_tiled_top1`), the $3B_{\text{row}}$ active nodes $(\text{tree_root}, \text{other}_0, \text{other}_1)$ are reconstructed simultaneously as contiguous columns of width $3B_{\text{row}}$ so that all 32 threads in a warp execute coalesced memory reads and writes during the bottom-up and top-down ancestor walks before a 16×16 shared-memory transpose into row-major `d_row_tmp`. Similarly, the $3(N-1)$ neighbor-clade entries nd_u are reconstructed directly from subdist_int_leaf without materializing subdist (`k_load_nd_u_leaves_tiled`, `k_reconstruct_nd_u_cols`) and exchanged across GPUs via peer-to-peer DMA (`cudaMemcpyPeerAsync`).

4.5 Exact Clade-Condensation Theorem for Multi-Million-Taxon Minibatch Optimization

To scale beyond the $O(N^2)$ memory ceiling of full-tree distance tables ($N = 380{,}000$ on 8× H100) to $N \in [10^6, 10^7]$ taxa on a single GPU or multi-GPU node, we use an algebraic property unique to Pauplin's dyadic BME weight $2^{-d_T(x,y)}$: any collection of $K = N_{\text{eff}}$ disjoint subtrees (clades) can be collapsed into single "super-leaves" such that the discrete gradient on the K-leaf condensed tree is identical to the full N-leaf BME discrete gradient for all moves among the active branches.

Theorem 4.5 (Exact Clade-Condensation and Minibatch Sufficiency for BME)**.** *Let T be a rooted binary tree on N leaves $\{0, \dots, N-1\}$ partitioned into $K = N_{\text{eff}}$ disjoint rooted subtrees $S_0, S_1, \dots, S_{K-1}$ rooted at tree nodes $r_0, r_1, \dots, r_{K-1}$ (where a fully expanded active leaf x is a singleton clade $S_k = \{x\}$ with $r_k = x$, while a frozen inactive clade S_k has $|S_k| \geq 2$). For each leaf $x \in S_k$, define its internal dyadic weight $\omega_x = 2^{-d_{S_k}(r_k, x)}$, where $d_{S_k}(r_k, x)$ is the number of internal nodes of S_k along the path from r_k to x (including r_k if $|S_k| \geq 2$, and $\omega_x = 1$ if $S_k = \{x\}$). Define the $K \times K$ condensed super-leaf distance matrix $\widetilde{D} \in \mathbb{R}^{K \times K}$ by:*

$$\widetilde{D}_{ab} = \begin{cases} \displaystyle\sum_{x \in S_a} \sum_{y \in S_b} \omega_x \, \omega_y \, D_{xy}, & \text{if } a \neq b, \\ 0, & \text{if } a = b. \end{cases} \tag{14}$$

Let $\widetilde{T}$ denote the K-leaf binary skeleton tree obtained by replacing each subtree S_k at r_k with a single super-leaf $k \in \{0, \dots, K-1\}$. Then:

1. ***Internal Weight Normalization (Kraft Identity):*** *Within every clade S_k, $\sum_{x \in S_k} \omega_x = 1$. If $D_{xy} = \langle \phi(x), \psi(y) \rangle$ is evaluated from sequence embeddings or k-mer/Hamming profiles*

$\phi(x) \in \mathbb{R}^P$, *each super-leaf* a *is represented by a single condensed profile* $\widetilde{\phi}(a) = \sum_{x \in S_a} \omega_x \phi(x)$ *computed in* $O(NP)$ *time (*`k_condense_super_profiles`*), reducing* $\widetilde{D}$ *construction to a* $K \times K$ *kernel (*`k_build_condensed_D`*).*

2. ***Additive Decomposition of Total BME Length:*** *For any skeleton topology* $\widetilde{T}$ *on the* K *super-leaves, the exact* N*-leaf Pauplin BME length* $L(T; D)$ *decomposes into the* K*-leaf BME length* $L(\widetilde{T}; \widetilde{D})$ *plus a topology-invariant internal constant* $C_{int}(S_0, \ldots, S_{K-1}; D)$ *that depends only on distances within individual clades* S_k*:*

$$L(T; D) = L(\widetilde{T}; \widetilde{D}) + \sum_{k=0}^{K-1} \sum_{\substack{x,y \in S_k \\ x<y}} D_{xy}\, 2^{-d_{S_k}(x,y)}. \tag{15}$$

3. ***Exact Minibatch Discrete Gradient Preservation:*** *For every coordinate perturbation* (i, j) *(*$0 \le j \le 2i$*,* $i \in \{1, \ldots, K-2\}$*) on the Phylo2Vec vector* $\widetilde{v} \in \mathcal{V}_K$ *of* $\widetilde{T}$ *(and its corresponding subtree move* $T \to T^{(i \leftarrow j)}$ *on the full* N*-leaf tree via* `expand_condensed_tree_to_v_c`*), the condensed discrete gradient equals the exact* N*-leaf discrete gradient to machine precision:*

$$\Delta L(\widetilde{T}^{(i \leftarrow j)}; \widetilde{D}) = L(\widetilde{T}^{(i \leftarrow j)}; \widetilde{D}) - L(\widetilde{T}; \widetilde{D}) \equiv L(T^{(i \leftarrow j)}; D) - L(T; D) = \Delta L(T^{(i \leftarrow j)}; D). \tag{16}$$

Proof. For any two leaves $x \in S_a$ and $y \in S_b$ in distinct clades ($a \neq b$), the unique path between x and y in T decomposes into the path from x to r_a inside S_a (containing $d_{S_a}(r_a, x)$ internal nodes), the skeleton path between super-leaves a and b in $\widetilde{T}$ (containing $d_{\widetilde{T}}(a, b)$ internal skeleton nodes), and the path from r_b to y inside S_b (containing $d_{S_b}(r_b, y)$ internal nodes). Therefore $d_T(x, y) = d_{S_a}(r_a, x) + d_{\widetilde{T}}(a, b) + d_{S_b}(r_b, y)$, so Pauplin's weight factors multiplicatively as $2^{-d_T(x,y)} = \omega_x\, 2^{-d_{\widetilde{T}}(a,b)}\, \omega_y$. Summing over all unordered pairs $\{x, y\}$ and separating inter-clade pairs ($a \neq b$) from intra-clade pairs ($a = b$) yields Eq. (15). Because any SPR move on $\widetilde{T}$ leaves the internal topologies of $S_0, \ldots, S_{K-1}$ unchanged, the intra-clade sum C_{int} cancels identically in $L(T^{(i \leftarrow j)}; D) - L(T; D)$. □

Remark 4.6 (Algorithmic Tradeoff: $2^{-\text{depth}}$ Signal Dilution, Capped Dyadic Surrogates, Exact Changed-Pair Acceptance, and Sequential Wave Outside-Profile Rebuilding)**.** While Theorem 4.5 guarantees that full-profile condensation (`condense_tree_bme_c` and `k_condense_super_profiles` + `k_build_condensed_D`) has zero approximation error for the condensed moves (ΔL is exact for each frozen clade moving as a unit), practical large-N minibatch optimization involves three design considerations:

1. **Search-Space Restriction and $2^{-\text{depth}}$ Signal Dilution:** If a single misplaced taxon x^* is buried at depth d inside a frozen clade S_b while its true sister clade S_a is unpacked, its weight inside the super-leaf profile $\widetilde{\phi}(b) = \sum_{y \in S_b} 2^{-d(r_b, y)} \phi(y)$ is attenuated by 2^{-d}, so the optimizer will not pull the entire clade S_b across the tree unless x^* itself is unpacked into the active minibatch; moving x^* from S_b to S_a when S_a and S_b are unpacked in separate minibatches likewise requires multiple passes. Our minibatch schedulers (`select_minibatch_clades_c` and `optimize_phylo2vec_bme_minibatch_c`) mitigate both effects by rotating the unpacking window across tree neighborhoods and retaining a backbone of condensed clade representatives.

2. **Best-First CSR Dyadic-Weighted Surrogate Profiles and Exact Changed-Pair Full-Objective Acceptance:** When distances come from a precomputed matrix D (`condense_tree_bme_c`) or linear inner-product embeddings $\phi(x) \in \mathbb{R}^P$ (`k_condense_super_profiles`, Table 7), exact dyadic profiles $\widetilde{\phi}(a) = \sum_{x \in S_a} \omega_x \phi(x)$ are computed over all N leaves. When pairwise distances are evaluated on the fly from non-linear sequence formulas such as Jukes–Cantor log-corrected Hamming distances (`bench_rnasim_gpu.cu`) or an

$O(\log N)$ LCA tree oracle (`bench_minibatch_recovery.cu`) without $O(N^2)$ distance storage, a naive unweighted 2^d-slot top-split table (duplicating leaves across $2^d \in \{4, 8\}$ uniform slots of weight 2^{-d}) is only exact when every leaf in the clade has depth $\leq d$; on unbalanced or caterpillar clades of size as small as $d+2$ (or 9 leaves for $2^d = 8$), uniform slot duplication distorts deep-leaf weights by up to $2^{\text{depth}-d}$ and can invert the sign of ΔL. Instead, `compute_clade_dyadic_profile` and `compute_outgroup_dyadic_profile` expand internal nodes in best-first order (highest dyadic weight $\omega = 2^{-d}$ first) and store explicit (leaf_id, ω_x) pairs in compressed sparse row (CSR) arrays (`d_prof_offsets`, `d_prof_leaves`, `d_prof_weights`) up to a capacity of $M_{\text{prof}} = 64$–128 leaves per super-leaf (`k_build_true_condensed_D` and `k_build_real_seq_condensed_D`), with `compute_outgroup_dyadic_profile` walking the full ancestor path from sub_root to the unrooted tree root (suppressing the degree-2 root split) for proper-subtree windows (sub_root $\neq$ root). Every condensed clade or outgroup of size $\leq M_{\text{prof}}$ (or of any size when uncapped, $M_{\text{prof}} \leq 0$)—whether balanced, unbalanced, or a caterpillar—has 100% of its internal nodes expanded and is represented with 100% of its leaves and their exact dyadic weights $\omega_x = 2^{-d(r_a,x)}$. For clades or outgroups whose size exceeds a finite cap ($|S_a| > M_{\text{prof}} > 0$), expansion stops at M_{prof} entries and collapses each remaining frontier subtree in the priority queue onto a single representative leaf `node_rep0[u]` carrying that subtree's mass: while on a pure caterpillar spine only one internal node of residual weight $2^{-(M_{\text{prof}}-1)} \leq 2^{-63} \ll \varepsilon_{\text{fp32}}$ remains unexpanded, on a balanced clade of size $|S_a| > M_{\text{prof}}$ stopping at depth $\log_2 M_{\text{prof}}$ leaves M_{prof} frontier subtrees of weight $1/M_{\text{prof}}$ each (retaining total-variation distance $1 - M_{\text{prof}}/|S_a|$ when collapsed onto single leaves; for instance, on a 4,102-leaf tree with a 4,096-leaf balanced clade A, an adversarial split metric marking the M_{prof} selected representatives of A yields an unrefined capped surrogate change of -0.25 versus a true full-tree change of $+0.2421875$ at $M_{\text{prof}} = 64$ and $+0.234375$ at $M_{\text{prof}} = 128$). Consequently, the inner SPR search loop over the capped matrix $\widetilde{D}^{\text{cap}}$ is an $O(M_{\text{prof}}^2)$ heuristic proposal mechanism above the cap (an improving full-tree move whose capped surrogate score is non-negative may not be proposed). To guarantee that no accepted skeleton update ever worsens the true full-tree BME objective $L(T; D)$, `optimize_skeleton_on_gpu` (and `run_subtree_minibatch_window` for both sub_root = root and sub_root $\neq$ root with outside profiles) performs an *exact changed-pair GPU acceptance check* before committing any candidate topology $\widetilde{T}_{\text{best}} \neq \widetilde{T}_{\text{init}}$: it identifies all super-leaf pairs (a, b) whose unrooted topological edge distance changed ($d_{\widetilde{T}_{\text{best}}}(a,b) \neq d_{\widetilde{T}_{\text{init}}}(a,b)$) and where a or b was capped, evaluates their exact uncapped super-leaf distance $\widetilde{D}^{\text{exact}}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_x \omega_y D(x, y)$ in FP64 on the GPU (`k_refine_true_condensed_D_pairs` and `k_refine_real_seq_condensed_D_pairs`) using the full uncapped clade and outgroup profiles ($\sum_{a=0}^{K-1} |S_a| = N$), computes the exact full-tree objective change

$$\Delta L_{\text{full}} = L(T_{\text{best}}; D) - L(T_{\text{init}}; D) = \sum_{\substack{0 \leq a < b < K \\ d_{\text{best}}(a,b) \neq d_{\text{init}}(a,b)}} \left(2^{1-d_{\text{best}}(a,b)} - 2^{1-d_{\text{init}}(a,b)}\right) \widetilde{D}^{\text{exact}}_{ab}, \quad (17)$$

and rejects the candidate update (restoring $\widetilde{T}_{\text{init}}$ and returning 0) whenever $\Delta L_{\text{full}} \geq -10^{-6}$.

3. **Cross-Window Dyadic Profile Coupling and Sequential Live Outside-Profile Rebuilding:** Even when two subtree windows u_A, u_B in a wave are node-disjoint, an internal rearrangement inside u_A alters its root-to-leaf dyadic weight vector by $\Delta p_A = p_A^{(1)} - p_A^{(0)}$. If u_A and u_B are evaluated simultaneously against frozen pre-wave outside profiles and merged in parallel, their joint full-tree BME change differs from the sum of their individual local changes by the cross-window interaction term $2^{1-d(u_A,u_B)} (\Delta p_A)^\top D_{L(u_A),L(u_B)} (\Delta p_B)$ (for example, on a 1,540-leaf sequence instance with two disjoint 768-leaf windows u_A, u_B, two individually improving moves with $\Delta L_A = -0.003295$ and $\Delta L_B = -0.025449$ have cross-window interaction $+0.079269$, so applying both against frozen outside profiles yields

a true full-tree change of $+0.050526$ despite a frozen sum of -0.028744). To preserve exact full-tree descent across every wave, `run_multi_gpu_minibatch_bme` commits windows sequentially, propagating ancestor summaries (`update_one_node_summary`) and rebuilding each window's outside profile (`compute_outgroup_dyadic_profile`) from the live updated tree immediately before optimization so that the cumulative objective change telescopes as $L(T^{(M)}; D) - L(T^{(0)}; D) = \sum_{m=1}^{M} \Delta L_m \leq 0$.

5 Software Architecture and Module Reference

The codebase is organized into modular C99, Objective-C/Metal, CUDA C++, and Python components:

- `phylo2vec_fastme.c` & `libphylo2vec_fastme.dylib`: Core C99 library (with overflow-safe `size_t` dynamic workspace allocation up to $N = 100{,}000$) providing:
  - `calculate_discrete_gradient_c`: Lower-triangular ($0 \leq j < i$) discrete gradient of shape $(N-1, N-2)$.
  - `calculate_discrete_gradient_full_c`: Complete ($0 \leq j \leq 2i$) discrete gradient of shape $(N-1, 2N-3)$ implementing Theorem 2.3.
  - `relabel_from_rooted_tree` & `relabel_cherry_first_c`: Top-down cherry-first gauge relabeling with clade-flip duality (flip_mode $\in \{0, 1\}$, Theorem 2.1).
  - `encode_rooted_to_v_c` & `encode_unrooted_to_v_c`: Exact inverse bijections from rooted or unrooted trees to $v \in \mathcal{V}$ (Theorem 2.2).
  - `optimize_phylo2vec_bme_c`: 4-stage Phylo2Vec BME tree optimizer with node-disjoint K-SPR batch updates, dual-mode gauge rotation, and 2-SPR basin hopping.
  - `condense_tree_bme_c`, `expand_condensed_tree_to_v_c`, `select_minibatch_clades_c`, & `optimize_phylo2vec_bme_minibatch_c`: Exact clade-condensed minibatch BME condensation, full-tree topological grafting, and rotating minibatch optimization implementing Theorem 4.5.
  - `optimize_fastme_bme_c`: Reference C99 implementation of FastME's unrooted `BMEAveragesTable` and `bmeSPR` topological hill-climber [2, 3].
- `discrete_gradient.py` & `compare_with_fastme_cli.py`: High-level Python/NumPy/`ctypes` interface wrapping universal (`arm64 + x86_64`) `libphylo2vec_fastme.dylib`, with input validation, path-compressed Union-Find ancestry decoding, and independent Newick round-trip BME verification.
- `verify_all.c`: Mathematical verification and regression harness testing Theorem 2.1, Theorem 2.2, and Theorem 2.3 against brute-force $O(N^3)$ Pauplin evaluations, fault-injection self-tests, $N = 6$ counterexamples, canonical unrooted split bipartition isomorphism, and $N \in \{1024, 1025, 1026\}$ boundary scaling under UndefinedBehaviorSanitizer (`-fsanitize=undefined`).
- `verify_spr_coverage.py`: Brute-force enumeration of the unrooted SPR neighborhood, measuring the fraction reached by single-coordinate moves in each cherry-first gauge and checking that optimizer outputs are exact SPR local optima.
- `bench_optimizers.c`: End-to-end benchmark suite linked directly against `phylo2vec_fastme.c`, comparing `optimize_phylo2vec_bme_c` against `optimize_fastme_bme_c` across additive ($\sigma = 0.00$) and log-normal noisy ($\sigma = 0.15$) phylogenetic distance matrices with win/tie/loss and mean/SD/median/IQR metrics.

- `bench_parallel_gpu.m`: Large-N ($N = 100 \dots 4{,}000$) multi-core CPU (Grand Central Dispatch), Column-Tiled low-memory CPU (Theorem 4.4), Apple M5 Metal GPU shader (with 16×16 threadgroup tiled transpose), and Zero-Copy Hybrid Unified Memory benchmark suite.

- `phylo2vec_cuda_8xh100.cu`: Multi-GPU NVIDIA CUDA C++ (`sm_90` / `sm_100`) implementation with lockstep column-parallel `subdist` kernels, 16×16 shared-memory transpose (`k_transpose_leaf_block`), race-free destination indexing (`next_k_same_lmin`), dynamic shared-memory SPR kernels (`k_spr_rows_smem`) with automatic global-memory fallback (`k_spr_rows_gmem`), fused top-1 row candidate reduction (`k_spr_rows_top1`), column-tiled autonomous row kernels (`k_tiled_leaf_cols` and `k_spr_rows_tiled`), and a barrier-accurate host SIMT emulator testing forward and reverse thread scheduling.

6 Empirical Evaluation and Scaling Analysis

6.1 Mathematical Exactness and Sanitizer Verification (`verify_all.c`)

Running `./verify_all` compiled with UndefinedBehaviorSanitizer and BoundsSanitizer (`-fsanitize=undefined,bounds -fno-sanitize-recover=all`) validates every mathematical identity and adversarial edge case against independent brute-force $O(N^3)$ evaluations of Pauplin's formula (Eq. (1)):

1. **Test 1 (Full $0 \le j \le 2i$ Discrete Gradient & Fault-Injection Self-Test):** Across 100 random trees ($N = 20$) and all 36,000 coordinate perturbations (i, j) with $0 \le j \le 2i$, the maximum absolute discrepancy between `calculate_discrete_gradient_full_c` and brute-force Pauplin evaluation is $\mathbf{3.553 \times 10^{-14}}$ (exact to IEEE 754 double-precision machine epsilon), while injected $+1.0$ and `NaN` faults are rejected.

2. **Test 2 (Repeated-Minimum-Leaf & $i = 1$ Root-Overlap Edge Cases):** Evaluates the $N = 6$ boundary topologies $v = [0, 0, 0, 0, 0]$ (star/caterpillar) and $v = [0, 1, 1, 1, 1]$ across all $2i + 1$ coordinates (including row $i = 1$, column $j = 1$), achieving $\mathbf{0.000 \times 10^{0}}$ error.

3. **Test 3 ($O(N)$ Cherry-First Gauge Relabeling Verification):** Across 200 general Phylo2Vec trees with $0 \le v_i \le 2i$ ($N = 25$) tested under both flip_mode $= 0$ and flip_mode $= 1$, 100% satisfy $v_i^{\mathrm{cf}} \le i$ for all i with valid bijective permutations $\pi \in S_N$, and the maximum BME length difference $|L(v^{\mathrm{cf}}; D_\pi) - L(v; D)|$ is $\mathbf{1.776 \times 10^{-14}}$ (mode 0) and $\mathbf{2.665 \times 10^{-14}}$ (mode 1).

4. **Test 4 (Exact Inverse Bijection & Canonical Split Isomorphism):** Across 1,000 general rooted vectors ($N = 40$) and 1,000 unrooted trees ($N = 40$), `encode_rooted_to_v_c` achieves **1000/1000** (**100.0**%) bit-for-bit recovery of $v \in \mathcal{V}$, and `encode_unrooted_to_v_c` achieves **1000/1000** (**100.0**%) exact recovery of all $N - 3$ canonical 64-bit unrooted split bipartitions and BME lengths ($\mathbf{0.000 \times 10^{0}}$ max diff).

5. **Test 5 (Large-N Buffer Boundary Verification at $N \in \{1024, 1025, 1026\}$):** Validates dynamic workspace allocation, cherry-first relabeling, rooted/unrooted inverse encoding, and end-to-end `optimize_phylo2vec_bme_c` execution past the $N = 1{,}024$ boundary with zero sanitizer traps.

6. **Test 6 (Exhaustive $(2N - 3)!!$ Tree Enumeration for $N = 4, 5, 6, 7$):** Enumerates all $15 + 105 + 945 + 10{,}395 = \mathbf{11{,}460}$ rooted binary trees across $N \in \{4, 5, 6, 7\}$, verifying 11,460/11,460 (100%) exact rooted/unrooted inverse bijections and dual-mode cherry-first gauge relabelings, and verifying all $6 + 24 + 120 + 720 = \mathbf{870}$ cherry-first trees (28,488 gradient entries) against brute-force Pauplin evaluation with max error $\mathbf{2.776 \times 10^{-15}}$.

7. **Test 7 (Adversarial Topologies & Degenerate Distance Matrices):** Tests pure left/right caterpillars ($v_i = 0$ and $v_i = i$), alternating trees, and balanced trees on all-zero matrices ($D_{xy} = 0$), constant matrices ($D_{xy} = 1$), ultrametric clock matrices, and extreme dynamic ranges ($D_{xy} \in [10^{-12}, 10^5]$) with scale-relative convergence tolerances $\varepsilon_{\text{tol}} = 10^{-12}(|L| + 1)$, achieving max relative error $\mathbf{1.255 \times 10^{-13}}$.

8. **Test 8 (Exact Clade-Condensed Minibatch BME Theorem & Optimizer Verification):** Across 50 random trees ($N = 40$ condensed to $N_{\text{eff}} = 16$ active super-leaves) and all 16,231 valid SPR coordinate perturbations (i, j) on the condensed trees, verifies that every condensed discrete gradient entry $\Delta L(\widetilde{T}^{(i \leftarrow j)}; \widetilde{D})$ matches the exact full-tree $N \times N$ Pauplin BME length change $\Delta L(T^{(i \leftarrow j)}; D)$ to $\mathbf{1.564 \times 10^{-13}}$ (machine epsilon), and verifies monotonic tree-length reduction in `optimize_phylo2vec_bme_minibatch_c` ($N = 80$, $N_{\text{eff}} = 24$, $L : 34.4322 \to 20.6694$).

Running the barrier-accurate SIMT verifier `./phylo2vec_cuda_verify` (compiled from `phylo2vec_cuda_8xh100.cu` with `-fsanitize=undefined,bounds`) under forward ($0 \ldots 127$), reverse ($127 \ldots 0$), and randomized Fisher–Yates per-barrier thread-order fuzzing across all synchronization phases confirms zero race conditions on the $N = 6$ counterexamples (err $= 0.000$) and $N = 16, 32$ fuzz trials (max_err $= 1.244 \times 10^{-6}$), and verifies that all four GPU kernel paths (`smem`, `gmem`, `top1`, and `tiled`) match the FP64 CPU reference across $N \in \{50, 100, 250, 500\}$ to within single-precision machine epsilon ($\leq \mathbf{6.35 \times 10^{-7}}$).

Finally, `verify_spr_coverage.py` checks SPR local optimality end to end. For 90 runs of `optimize_phylo2vec_bme_c` from random starting vectors ($N \in \{10, 16, 24\}$, 15 runs each, on additive tree distances with log-normal noise $\sigma \in \{0.15, 0.5\}$), every one of the $2(N-3)(2N-7)$ unrooted SPR neighbors of the output was scored by an independent Pauplin evaluation: in **90/90** runs no neighbor improves on the output, and the reported BME length matches the independent evaluation. Every single-coordinate move in either gauge was also confirmed to be exactly one unrooted SPR move.

6.2 End-to-End Topological Optimization vs. FastME (`bench_optimizers.c`)

We evaluated end-to-end tree reconstruction from identical random starting trees across $N \in \{20, 50, 100, 150, 200\}$ under two regimes:

1. **Benchmark 1 (Noiseless Additive Distance Matrices, $\sigma = 0.00$):** We generated ground-truth trees T^* with uniform branch lengths $w_e \sim \text{Uniform}(0.05, 0.55)$ and exact additive pairwise distances $D_{xy} = \sum_{e \in P_{T^*}(x,y)} w_e$, where the unique global minimum of the BME objective is T^* (RF $= 0$).

2. **Benchmark 2 (Noisy Phylogenetic Distance Matrices, $\sigma = 0.15$):** We applied symmetric log-normal estimation noise $\tilde{D}_{xy} = D_{xy} \exp(\sigma Z_{xy})$, $Z_{xy} \sim \mathcal{N}(0, 1)$, to create rugged BME landscapes.

Tables 1 and 2 report the mean final BME tree length $L(T)$, strict wins / exact ties / losses (Win/Tie/Loss at tolerance 10^{-9}) of Phylo2Vec vs. FastME, normalized Robinson-Foulds topological error (RF $\in [0, 1]$) against T^*, mean number of $O(N^2)$ SPR iterations/gradient evaluations, single-core wall-clock runtime (mean $\pm$ SD and median[IQR] in milliseconds), and mean wall-clock speedup.

Across all 60/60 noiseless trials ($\sigma = 0.00$), the Phylo2Vec BME optimizer recovers the exact ground-truth topology (RF $= 0.0000$, 100% ties with FastME) while requiring up to **3.32**$\times$ fewer $O(N^2)$ evaluations (63.6 vs. 211.0 at $N = 200$) and achieving up to a **4.16**$\times$ single-core wall-clock speedup on Apple M5 (18.14 ms vs. 75.46 ms) and a **5.74**$\times$ single-core speedup on AMD Ryzen Threadripper PRO 5995WX (27.96 ± 1.18 ms vs. 160.64 ± 5.51 ms at $N = 200$). Across all 60 noisy trials ($\sigma = 0.15$), Phylo2Vec achieves 2 strict wins, 57 exact ties, and 1 loss (98.3% win/tie rate;

Table 1: Benchmark 1: Noiseless Additive Tree Distances ($\sigma = 0.00$, Exact Ground-Truth Recovery).

N	FastME L	P2V L	Win/Tie/Loss	FME RF	P2V RF	FME Iters	P2V Evals	FME (ms)	P2V (ms)	P2V Med [IQR]	Speedup
20	11.12739	11.12739	**0 / 20 / 0**	0.0000	0.0000	13.4	11.2	0.12 ± 0.02	0.08 ± 0.01	0.07 [0.02]	**1.54**×
50	29.42572	29.42572	**0 / 20 / 0**	0.0000	0.0000	42.2	19.4	1.58 ± 0.35	0.57 ± 0.13	0.59 [0.16]	**2.78**×
100	59.52693	59.52693	**0 / 10 / 0**	0.0000	0.0000	97.7	35.4	9.05 ± 0.84	2.63 ± 0.26	2.65 [0.44]	**3.44**×
150	89.04365	89.04365	**0 / 5 / 0**	0.0000	0.0000	156.2	49.4	31.47 ± 0.77	7.92 ± 0.86	7.75 [0.64]	**3.97**×
200	116.32398	116.32398	**0 / 5 / 0**	0.0000	0.0000	211.0	63.6	75.46 ± 2.42	18.14 ± 0.74	18.05 [1.17]	**4.16**×

Table 2: Benchmark 2: Noisy Phylogenetic Distances ($\sigma = 0.15$, Log-Normal Multiplicative Noise).

N	FastME L	P2V L	Win/Tie/Loss	FME RF	P2V RF	FME Iters	P2V Evals	FME (ms)	P2V (ms)	P2V Med [IQR]	Speedup
20	11.19777	11.19777	**0 / 20 / 0**	0.0500	0.0500	14.4	12.4	0.07 ± 0.01	0.05 ± 0.01	0.04 [0.02]	**1.60**×
50	29.25507	29.25607	**1 / 18 / 1**	0.0660	0.0649	46.1	25.9	1.07 ± 0.07	0.49 ± 0.09	0.49 [0.10]	**2.18**×
100	59.25471	59.25471	**0 / 10 / 0**	0.0526	0.0526	102.6	41.0	10.62 ± 2.53	3.32 ± 0.56	3.18 [0.52]	**3.19**×
150	92.14846	92.14846	**0 / 5 / 0**	0.0395	0.0395	166.0	61.0	33.97 ± 1.01	10.15 ± 1.35	10.17 [1.35]	**3.35**×
200	120.94220	120.94220	**1 / 4 / 0**	0.0467	0.0487	235.8	84.4	85.16 ± 2.06	24.67 ± 2.79	24.51 [4.48]	**3.45**×

119/120 = 99.2% overall across both benchmarks) with a **3.45**× speedup at $N = 200$ on Apple M5 (24.67 ms vs. 85.16 ms) and a **4.93**× speedup on Threadripper PRO 5995WX (36.91 ± 3.94 ms vs. 181.97 ± 4.30 ms). Validation against the external `fastme` 2.1.6.3 CLI binary (`compare_with_fastme_cli.py`, invoked via `fastme -i D.phy -u init.nwk -s -T 1`) confirms that our in-memory C99 `optimize_fastme_bme_c` implementation reproduces the exact final unrooted topology ($\mathrm{RF}(\mathrm{CLI}, \mathrm{FME}) = 0.0000$) and BME tree length ($|L_{\mathrm{CLI}} - L_{\mathrm{FME}}| < 10^{-8}$) across all $N \in \{20, 50, 100, 150, 200\}$.

6.3 Multi-Core CPU, Column-Tiled, and Metal GPU Scaling (`bench_parallel_gpu.m`)

Table 3 reports empirical scaling from $N = 100$ to $N = 4{,}000$ on an Apple M5 (10-core CPU + Metal GPU with Unified Memory), comparing: (1) 1-Core CPU FP64, (2) 10-Core Untiled CPU FP64 ($4N^2$ `subdist` footprint), (3) 10-Core Column-Tiled CPU FP64 ($1N^2$ `subdist_int_leaf` footprint, Theorem 4.4), (4) Complete 1-Core `subdist` DP (in both FP64 and FP32, including the 3-way root row and diagonal zeroing) vs. Metal GPU FP32 `subdist` DP (including 16×16 threadgroup tiled transpose), and (5) Zero-Copy Hybrid GPU+CPU Unified Memory execution, along with maximum absolute errors against the 1-Core FP64 reference for both FP64 and FP32 paths.

Table 3: Large-N Discrete Gradient Scaling on Apple M5 ($N = 100 \ldots 4{,}000$).

N	1C F64	10C F64	10C Spd	Tiled F64	1C sub(F64)	1C sub(F32)	GPU sub(F32)	GPU Spd	Hybrid F32	Hyb Spd	F64 Diff	F32 Diff
100	0.055 ms	0.068 ms	0.80×	0.096 ms	0.012 ms	0.011 ms	0.517 ms	0.02×	0.524 ms	0.10×	0.00e+00	4.35e-07
250	0.332 ms	0.322 ms	1.03×	0.246 ms	0.076 ms	0.058 ms	0.906 ms	0.08×	0.934 ms	0.36×	0.00e+00	5.24e-07
500	1.273 ms	0.433 ms	**2.94**×	0.508 ms	0.312 ms	0.223 ms	1.216 ms	0.26×	1.742 ms	0.73×	5.55e-17	6.63e-07
1,000	5.860 ms	1.878 ms	**3.12**×	2.081 ms	1.565 ms	1.344 ms	2.699 ms	0.58×	3.097 ms	**1.89**×	4.44e-16	8.11e-07
2,000	27.399 ms	10.230 ms	**2.68**×	10.806 ms	8.954 ms	9.533 ms	4.462 ms	**2.01**×	6.312 ms	**4.34**×	4.44e-16	8.93e-07
3,000	85.815 ms	24.428 ms	**3.51**×	**23.777 ms**	22.504 ms	35.030 ms	8.970 ms	**2.51**×	14.359 ms	**5.98**×	5.27e-16	9.84e-07
4,000	132.225 ms	51.265 ms	**2.58**×	**43.329 ms**	68.188 ms	59.289 ms	7.571 ms	**9.01**×	26.922 ms	**4.91**×	5.00e-16	1.13e-06

At $N = 3{,}000$ and $N = 4{,}000$, the Column-Tiled FP64 implementation (`Tiled F64`, Theorem 4.4) is both **4**× smaller in `subdist` memory (128 MB vs. 512 MB at $N = 4{,}000$) and **15**% faster (43.33 ms vs. 51.27 ms) than the untiled 10-core CPU kernel because reconstructing the three active rows per SPR scan inside L1/L2 cache avoids 512 MB DRAM cache misses. On the Metal GPU, the lockstep column-parallel `subdist` kernel achieves up to a **9.01**× speedup over 1-Core FP64 `subdist` (**7.83**× over 1-Core FP32 `subdist`) and up to a **5.98**× end-to-end hybrid speedup.

6.4 Empirical Multi-GPU Scaling on 2x NVIDIA RTX A6000 and 8x NVIDIA H100 80GB HBM3

We benchmarked `phylo2vec_cuda_8xh100.cu` on two multi-GPU Linux systems:

1. **Workstation Server (2× NVIDIA RTX A6000, `sm_86`):** AMD Ryzen Threadripper PRO 5995WX (64 cores / 128 threads, 251 GiB RAM) with two NVIDIA RTX A6000 GPUs (48 GB GDDR6 each, 84 SMs, 100 KB shared memory/SM).

2. **Gefion DGX H100 Node `dgx158` (8× NVIDIA H100 80GB HBM3, `sm_90`):** Dual Intel Xeon Platinum 8480+ node with eight NVIDIA H100 SXM5 GPUs (79.2 GB usable HBM3 each = 633.6 GB total HBM3, 132 SMs/GPU = 1,056 SMs total, 228 KB opt-in dynamic shared memory/SM via `cudaFuncAttributeMaxDynamicSharedMemorySize`, and 900 GB/s bidirectional NVLink 4.0 peer-to-peer interconnect).

Table 4 reports single-GPU kernel timings and numerical accuracy across $N \in \{100, 250, 500, 1{,}000, 2{,}000, 4{,}000\}$ on both 1× NVIDIA RTX A6000 and 1× NVIDIA H100 80GB HBM3 against the single-core FP64 CPU reference (`calculate_discrete_gradient_full_c`). Across all sizes up to $N = 4{,}000$, all GPU kernel paths match the FP64 CPU reference to within $\leq 1.11 \times 10^{-6}$, and reach up to a **62.48**× end-to-end single-GPU speedup on 1× RTX A6000 (11.05 ms vs. 690.58 ms) and up to a **112.78**× single-GPU speedup on 1× H100 (6.22 ms vs. 701.38 ms at $N = 4{,}000$).

Table 4: Single-GPU Verification & Kernel Timings on 1× NVIDIA RTX A6000 (48 GB, `sm_86`) and 1× NVIDIA H100 (80 GB HBM3, `sm_90`) ($N = 100 \ldots 4{,}000$).

N	A6000 sub+smem	A6000 Spd	A6000 Tiled	H100 CPU Ref	H100 sub	H100 smem	H100 Total	H100 Spd	H100 Tiled	Max Diff	Status
100	0.061 ms	**8.44**×	0.078 ms	0.249 ms	0.044 ms	0.027 ms	0.071 ms	**3.51**×	0.095 ms	3.68e-07	**PASS**
250	0.186 ms	**15.92**×	0.207 ms	1.964 ms	0.116 ms	0.048 ms	0.164 ms	**11.98**×	0.205 ms	6.22e-07	**PASS**
500	0.382 ms	**32.95**×	0.511 ms	8.176 ms	0.227 ms	0.077 ms	0.304 ms	**26.89**×	0.460 ms	6.48e-07	**PASS**
1,000	1.035 ms	**33.69**×	1.988 ms	32.336 ms	0.661 ms	0.105 ms	0.766 ms	**42.21**×	1.030 ms	9.29e-07	**PASS**
2,000	3.851 ms	**40.15**×	7.288 ms	154.091 ms	1.557 ms	0.726 ms	2.283 ms	**67.50**×	4.273 ms	1.11e-06	**PASS**
4,000	11.053 ms	**62.48**×	43.124 ms	701.375 ms	3.565 ms	2.654 ms	6.219 ms	**112.78**×	22.327 ms	1.07e-06	**PASS**

Table 5 compares our large-N ($N \in \{1{,}000, 2{,}000, 4{,}000, 10{,}000, 20{,}000, 40{,}000\}$) multi-GPU scaling across 1×/2× RTX A6000 and 1×/8× H100 80GB HBM3 for both the untiled Full-`subdist` + `k_spr_rows_top1` pipeline and the low-memory Column-Tiled + `k_spr_rows_tiled_top1` pipeline:

- **228 KB Shared-Memory Advantage on H100:** Because H100 provides 228 KB of dynamic shared memory per SM (vs. 100 KB on RTX A6000), the per-row edge accumulator s_edge$[0 \ldots 2N-3]$ ($8N - 8$ bytes) remains in fast on-chip SRAM up to $N = 28{,}000$ on H100 (whereas it spills to global memory above $N = 12{,}400$ on A6000). Consequently, at $N = 20{,}000$, 1× H100 Full is **5.22**× faster than 1× RTX A6000 (155.93 ms vs. 813.29 ms).

- **8× H100 Multi-GPU Scaling at $N = 40{,}000$ (3.2×10^9 moves):** On 8× H100, the untiled Full-`subdist` pipeline (31.1 GB VRAM) completes the 3.2×10^9-move sweep in **158.36** ms (**5.15**× end-to-end speedup over 1× H100 at 816.34 ms, and a **7.70**× speedup on the Stage-3 SPR sweep from 756.26 ms down to 98.28 ms), while the 2.3× lower-memory Column-Tiled pipeline (13.6 GB VRAM) completes in **1.033** s (**4.62**× speedup over 1× H100 at 4.768 s and **5.45**× faster than 2× RTX A6000 at 5.629 s).

To push beyond single-GPU memory limits up to the aggregate 633.6 GB HBM3 capacity of the 8× H100 node, we implemented a distributed column-sharded pipeline (`run_sharded_multi_gpu_top1`) in `phylo2vec_cuda_8xh100.cu`. By Theorem 4.1 (Column Independence) and Theorem 4.4 (Column Tiling), the $(N-1) \times N$ table subdist_int_leaf can be partitioned by leaf columns across all $G = 8$ GPUs so that GPU g stores only its contiguous leaf-column shard $[c_{\text{start}}(g), c_{\text{end}}(g))$ of size $4N(N-1)/G \approx 0.5N^2$ bytes (8× smaller per GPU than replicated column tiling and 32× smaller

Table 5: Large-N & Multi-GPU Scaling Sweep on 1×/2× NVIDIA RTX A6000 and 1×/8× NVIDIA H100 80GB HBM3 ($N = 1{,}000 \ldots 40{,}000$).

N	2×A6000 Full	2×A6000 Tiled	1×H100 Full (sub + spr)	8×H100 Full	H100 Full Spd	Full VRAM	1×H100 Tiled	8×H100 Tiled	Tiled VRAM	Top-1 Match
1,000	0.93 ms	5.75 ms	0.76 ms [0.65 + 0.11]	0.79 ms	0.96×	27 MB	1.46 ms	4.47 ms	39 MB	**PASS** (0.00e+00)
2,000	4.89 ms	13.90 ms	2.21 ms [1.47 + 0.74]	1.71 ms	**1.29**×	92 MB	6.14 ms	5.21 ms	93 MB	**PASS** (2.38e-07)
4,000	14.69 ms	56.85 ms	6.01 ms [3.57 + 2.44]	3.98 ms	**1.51**×	336 MB	30.78 ms	10.98 ms	247 MB	**PASS** (0.00e+00)
10,000	90.45 ms	249.59 ms	29.61 ms [10.10 + 19.50]	13.03 ms	**2.27**×	1,985 MB	250.69 ms	57.53 ms	1,076 MB	**PASS** (0.00e+00)
20,000	422.73 ms	1,212.27 ms	155.93 ms [23.23 + 132.70]	41.06 ms	**3.80**×	7,785 MB	1,123.43 ms	244.13 ms	3,677 MB	**PASS** (4.77e-07)
40,000	*OOM*	5,628.62 ms	816.34 ms [60.08 + 756.26]	**158.36** ms	**5.15**×	31,142 MB	4,768.31 ms	**1,032.83** ms	13,614 MB	**PASS** (0.00e+00)

per GPU than untiled subdist). During Stage 2, each GPU computes 100% of its own leaf-column shard of subdist_int_leaf in parallel. During Stage 3, for each row tile $[u_{\text{start}}, u_{\text{end}})$ of internal nodes, each GPU g computes its local slice of the autonomous vectors nd_u$[b, c]$ and auto$[b, k, c]$ for $c \in [c_{\text{start}}(g), c_{\text{end}}(g))$ and exchanges the resulting $O(B_{\text{tile}} \cdot N/G)$ leaf slices across NVLink 4.0 via direct peer-to-peer DMA (`cudaMemcpyPeerAsync` with `cudaDeviceEnablePeerAccess`) before launching `k_spr_rows_tiled_top1` on its assigned subset of rows. For $N \leq 250{,}000$, each GPU also holds its $0.5N^2$-byte column shard of D in HBM3 (`Shard+Calib`); for $N \geq 350{,}000$, D tiles are generated on the fly per column batch (`StreamD+Calib`) so that almost all of the 79.2 GB HBM3 per GPU is dedicated to subdist_int_leaf.

Table 6 lists the measurements on 8× NVIDIA H100 80GB HBM3 from $N = 40{,}000$ (3.20×10^9 moves) up to the HBM3 limit at $N = 380{,}000$ (2.89×10^{11} candidate moves, 72,946 MB/GPU = 569.9 GB aggregate HBM3), where a full Stage-2 DP + Stage-3 sweep completes in **205.45** s with verified top-1 accuracy. At $N = 400{,}000$ the run exceeds GPU memory, as it requires 80,600 MB/GPU > 79.2 GB.

Table 6: Distributed Column-Sharded Scaling on 8× NVIDIA H100 80GB HBM3 (`dgx158`, $N = 40{,}000 \ldots 400{,}000$).

Tree Size (N)	Moves / Sweep ($2N^2$)	Stage 2 100% DP (ms)	Stage 3 Sweep (ms)	Total 8×H100 Sweep (s)	VRAM / GPU	Total 8×H100 VRAM	Mode & Verification Status
40,000	3.20×10^9	514.54 ms	517.92 ms	**1.032** s	2,386 MB	18.6 GB	`Full-Shard` [**PASS**]
80,000	1.28×10^{10}	2,112.82 ms	2,321.28 ms	**4.434** s	7,824 MB	61.1 GB	`Shard+Calib` [**PASS**]
120,000	2.88×10^{10}	4,785.75 ms	4,803.60 ms	**9.589** s	16,314 MB	127.5 GB	`Shard+Calib` [**PASS**]
180,000	6.48×10^{10}	10,848.67 ms	13,977.73 ms	**24.826** s	34,770 MB	271.6 GB	`Shard+Calib` [**PASS**]
250,000	1.25×10^{11}	39,981.55 ms	43,721.48 ms	**83.703** s	62,296 MB	486.7 GB	`Shard+Calib` [**PASS**]
350,000	2.45×10^{11}	76,240.05 ms	96,515.83 ms	**172.756** s	62,180 MB	485.8 GB	`StreamD+Calib` [**PASS**]
380,000	2.89×10^{11}	89,992.00 ms	115,457.57 ms	**205.450** s	72,946 MB	569.9 GB	`StreamD+Calib` [**PASS**]
400,000	3.20×10^{11}	*Hardware HBM3 Capacity Ceiling (requires* 80,600 *MB/GPU >* 79.2 *GB usable)*			80,600 MB	629.7 GB	*OOM Boundary Verified*

To scale beyond $N = 380{,}000$ to multi-million-taxon phylogenies ($N \in \{1{,}000{,}000, 5{,}000{,}000, 10{,}000{,}000\}$), we evaluated our GPU Clade-Condensed Minibatch pipeline (Theorem 4.5, `k_condense_super_profiles` + `k_build_condensed_D` + `run_multi_gpu_top1`) with active minibatch size $N_{\text{eff}} = 20{,}000$ (15,000 fully expanded leaves +5,000 condensed backbone clades per batch, 8×10^8 candidate SPR moves evaluated per batch) on both 2× NVIDIA RTX A6000 and 8× NVIDIA H100 (`dgx158`). Table 7 reports the empirical $O(N_{\text{total}})$ GPU profile-condensation + $N_{\text{eff}} \times N_{\text{eff}}$ condensed distance matrix construction time, the active $N_{\text{eff}} = 20{,}000$ multi-GPU BME discrete gradient sweep time, the number of batches per full N_{total}-taxon epoch ($\lceil N_{\text{total}}/15{,}000 \rceil$), total wall-clock time per full epoch across all N_{total} taxa, and peak VRAM per GPU. At $N_{\text{total}} = 10{,}000{,}000$ (10M taxa), an epoch across all 667 minibatches takes **39.20** s on 8× NVIDIA H100 (**199.35** s on 2× RTX A6000) and uses **7,862** MB (**7.8** GB) of VRAM per GPU, which shows that 10M-taxon BME optimization fits inside a single GPU's memory.

Table 7: Clade-Condensed Minibatch Scaling at $N_{\text{total}} = 1{,}000{,}000 \ldots 10{,}000{,}000$ Taxa ($N_{\text{eff}} = 20{,}000$) on 2× NVIDIA RTX A6000 and 8× NVIDIA H100 (`dgx158`).

Total Taxa (N_{total})	Active N_{eff}	Batches / Epoch	2×A6000 Cond+Grad	2×A6000 Epoch	8×H100 Cond (ms)	8×H100 Grad (ms)	8×H100 Epoch (s)	VRAM / GPU	Verification
1,000,000	20,000	67	4.45 + 284.44 ms	19.36 s	2.69 ms	48.53 ms	**3.43** s	7,793 MB	**PASS** (1.4e-05)
5,000,000	20,000	334	8.42 + 283.02 ms	97.34 s	4.86 ms	48.48 ms	**17.82** s	7,824 MB	**PASS** (2.1e-06)
10,000,000	20,000	667	17.29 + 281.58 ms	199.35 s	10.20 ms	48.57 ms	**39.20** s	7,862 MB	**PASS** (2.2e-06)

Finally, to verify ground-truth topological recovery at $N \in \{10{,}000, 50{,}000, 100{,}000, 500{,}000, 1{,}000{,}000\}$ without allocating an $O(N^2)$ distance matrix and with zero leakage of perturbation

metadata, we implemented a 100% blind GPU recovery benchmark (`bench_minibatch_recovery.cu`). The optimizer receives only the corrupted topology and a black-box $O(N)$-memory GPU Heavy-Light Decomposition (HLD) leaf-distance oracle that evaluates $d_{T^*}(x, y) = d_{\text{root}}(x) + d_{\text{root}}(y) - 2d_{\text{root}}(\text{LCA}_{T^*}(x, y))$ in $O(\log N)$ hops across CSR dyadic-weighted clade profiles (`compute_clade_dyadic_profile` and `k_build_true_condensed_D`, Remark 4.6). Corrupted clades are discovered blindly on the GPU by scanning all $N - 2$ internal edges for four-point condition quartet violations (`k_detect_quartet_violations`), and cross-clade partner windows are discovered blindly via GPU k-NN leaf-to-leaf distance queries (`k_find_knn_partner_clade`), tracked via invariant leaf anchors across skeleton graft-backs. Table 8 reports both intra-clade topological recovery (800 SPR perturbations inducing 2,457–2,816 wrong bipartition splits) and long-range cross-clade SPR recovery comparing Blind Naive Single-Clade Windows against Blind k-NN Paired-Clade Minibatching (Remark 4.6). Across $N \in \{10{,}000 \dots 1{,}000{,}000\}$, our blind GPU Clade-Condensed Minibatch optimizer recovers **96.98**%–**99.47**% of all corrupted bipartition splits (**93.06**%–**98.56**% canonical Phylo2Vec Hamming recovery) in 0.60–13.02 s, while Blind k-NN Paired-Clade Minibatching lifts cross-clade recovery from 74.70%–79.22% (naive single-clade windows) to **94.16**%–**100.00**% (reaching **100.00**% with 0 wrong splits at $N = 100{,}000$ and $N = 1{,}000{,}000$).

Table 8: 100% Blind Ground-Truth Topological Tree Recovery Across $N = 10{,}000 \dots 1{,}000{,}000$ Taxa (`bench_minibatch_recovery.cu`): Canonical Phylo2Vec Hamming Error $d_H(v, v^*)$, Exact Robinson–Foulds (RF) Wrong Splits, and Blind Naive vs. Blind k-NN Paired-Clade Recovery on Cross-Clade SPR Errors.

Taxa (N)	Window N_{eff}	Init → Final $d_H(v, v^*)$	Init → Final RF	d_H Recov %	RF Recov %	Time (s)	Cross-Clade Init RF	Naive Window Recov	Blind k-NN Guided Recov
10,000	629	766 → 11	2,650 → 14	**98.56**%	**99.47**%	0.60 s	1,439	77.14% (329 rem)	**94.16**% (84 rem)
50,000	692	780 → 50	2,457 → 70	**93.59**%	**97.15**%	0.92 s	1,680	74.70% (425 rem)	**98.45**% (26 rem)
100,000	808	778 → 16	2,609 → 21	**97.94**%	**99.20**%	1.19 s	1,511	79.22% (314 rem)	**100.00**% (0 rem)
500,000	1,310	773 → 29	2,599 → 44	**96.25**%	**98.31**%	6.52 s	1,539	75.24% (381 rem)	**95.71**% (66 rem)
1,000,000	1,346	778 → 54	2,816 → 85	**93.06**%	**96.98**%	13.02 s	1,516	79.22% (315 rem)	**100.00**% (0 rem)

To validate our optimizer on sequence alignments with all simulation shortcuts removed (no HLD oracle, no synthetic embeddings, and zero knowledge of the ground-truth topology T^*), we conducted two end-to-end phylogenetic reconstruction benchmarks against the two leading CPU Minimum-Evolution inference tools: **`FastME 2.1.6`** [3] (64 OpenMP threads, running `BIONJ + BME-NNI + BME-SPR` as well as refining a shared starting tree via `-u`) and **`VeryFastTree 4.0.5`** [6] (64 OpenMP threads, AVX2, running `VFT-InitNJ` and `VFT-ME -noml`).

First, we benchmarked all three methods on 1,596-bp DNA alignments simulated under the exact continuous-time Jukes–Cantor (`JC69`) Markov process along ground-truth subtrees ($N \in \{1{,}000, 2{,}500, 5{,}000, 10{,}000\}$) induced from the **RNASim** [5] phylogeny. Table 9 demonstrates two key results: (1) `FastME 2.1.6`'s exact CPU BME-SPR search scales cubically, jumping from 11.77 s at $N = 1{,}000$ to 286.12 s (4.8 min) on 64 CPU threads at $N = 2{,}500$ (24.3× slower for a 2.5× increase in N), whereas our 1-GPU Full-Tree Phylo2Vec BME optimizer completes $N = 2{,}500$ in **0.07** s (**4,087**× faster than 64-thread `FastME`) and achieves fewer Robinson–Foulds wrong splits (245 vs. 247); and (2) across every tested scale $N \in \{1{,}000, 2{,}500, 5{,}000, 10{,}000\}$, our GPU Phylo2Vec BME optimizer achieves the lowest RF topological error of any method (102 vs. 105 and 107 at $N = 1{,}000$; 245 vs. 247 at $N = 2{,}500$; 436 vs. 445 at $N = 5{,}000$; and 819 vs. 834 at $N = 10{,}000$, running **196** × –**666**× faster than 64-thread `VeryFastTree` and up to **4,769**× faster than 64-thread `FastME`).

Table 9: Controlled Jukes–Cantor (`JC69`) 1,596-bp Sequence Simulation Benchmark: 64-Thread `FastME 2.1.6` and `VeryFastTree 4.0.5` vs. 1× RTX A6000 GPU Phylo2Vec BME Optimizer.

Taxa (N)	Splits	VFT-InitNJ 64T	FastME 2.1.6 64T (BIONJ+SPR)	FastME 2.1.6 64T (-u InitNJ+SPR)	VFT-ME 64T (-noml)	GPU Full-Tree BME	GPU Minibatch BME	Speedup vs. FastME / VFT-ME
1,000	997	138 (13.84%, 0.64 s)	105 (10.53%, 11.77 s)	105 (10.53%, +14.04 s)	107 (10.73%, +6.66 s)	**105** (10.53%, **0.01** s)	**102** (**10.23**%, **0.01** s)	**1,177**× / **666**×
2,500	2,497	312 (12.49%, 1.44 s)	247 (9.89%, 286.12 s)	248 (9.93%, +220.26 s)	245 (9.81%, +16.28 s)	**245** (**9.81**%, **0.07** s)	249 (9.97%, **0.06** s)	**4,087**× / **233**×
5,000	4,997	621 (12.43%, 3.04 s)	> 45 *min (CPU wall)*	> 35 *min (CPU wall)*	445 (8.91%, +32.38 s)	443 (8.87%, 0.28 s)	**436** (**8.73**%, **0.15** s)	— / **216**×
10,000	9,997	1,285 (12.85%, 6.95 s)	> 6 *hr (CPU wall)*	> 5 *hr (CPU wall)*	834 (8.34%, +64.58 s)	825 (8.25%, 3.41 s)	**819** (**8.19**%, **0.33** s)	— / **196**×

Second, we evaluated all methods on the empirical biophysical **RNASim** 1,596-bp 16S

rRNA alignments [5] ($N \in \{1{,}000, 5{,}000, 10{,}000, 50{,}000, 100{,}000\}$ sequences evolved under thermodynamic secondary-structure folding selection, indels, and strong site-to-site rate heterogeneity). In addition to gap-aware `JC69` distances, we evaluated our GPU Bayesian Eigenvalue GTR + $\Gamma(\alpha, \alpha)$ distance estimator (`phylo2vec_eigen_dist_cuda.cu` [7]), which fits the 7-parameter GTR + Γ exchangeabilities $\hat{S} = [0.82, 1.14, 1.72, 0.39, 1.13, 0.80]$ and shape $\hat{\alpha}$ in 0.18–0.25 s on the GPU and computes reversible symmetrized Iteratively Reweighted WLS distances (`Eigen-RevSymWLS`). For large-N minibatch optimization (`bench_rnasim_gpu.cu`), our Subtree-Wave Multigrid Minibatch algorithm uses best-first CSR dyadic-weighted clade and outgroup profiles (`compute_clade_dyadic_profile`, `compute_outgroup_dyadic_profile`, and `k_build_real_seq_condensed_D`, Remark 4.6), exact changed-pair full-profile GPU acceptance verification (`k_refine_real_seq_condensed_D_pairs`), and priority-queue-bounded skeleton extraction (max_internals = 1,200, guaranteeing $K \leq 1{,}201 \leq 2{,}048$ even on maximally unbalanced caterpillar trees) across three sequential phases on a single GPU stream: (i) a global backbone pass at the tree root ($B_{\text{thresh}} = N/700$), (ii) a bottom-up connected-component wave (`build_component_waves`, partitioning all $N - 1$ internal nodes into connected components of $\leq 1{,}050$ internal nodes and scheduling all leaf-bearing components with $c_{\text{thresh}} = 1$ so that 100% of the N original leaves are uncondensed) anchored by an outgroup super-leaf whose CSR dyadic profile is rebuilt from the live updated tree immediately before each window (`compute_outgroup_dyadic_profile`), and (iii) a staggered medium cross-clade wave (first_cap = 525, max_cap = 1,050) with live ancestor summary propagation (`update_one_node_summary`). Notably, at $N = 1{,}000$, even when initialized from a **100% uniformly random binary tree** (997/997 wrong splits, zero NJ initialization), our GPU Full-Tree BME optimizer converges in **0.48** s ($\Delta L = -143.88$) to **178**/**997** wrong splits (17.85%), matching 64-thread `VeryFastTree` (178/997 in 7.46 s) and 64-thread `FastME -u` (178/997 in 11.85 s) from pure white noise. Starting from the shared `VFT-InitNJ` topology, Table 10 shows that our GPU Phylo2Vec BME optimizer outperforms both 64-thread `FastME 2.1.6` and 64-thread `VeryFastTree 4.0.5` in topological accuracy across **every** dataset size up to $N = 100{,}000$ while running **5.7** × –**115**× faster than `VeryFastTree` refinement (and up to **2**,**168**× faster than `FastME`): at $N = 50{,}000$, GPU Minibatch BME with exact changed-pair acceptance and sequential live outside-profile rebuilding reduces wrong splits from 6,729 to **4**,**950** (**9.90**% nRF, beating `VeryFastTree` by **30** splits) in **16.84** s vs. +313.53 s (**18.6**× faster refinement, or **22.4**× faster than `VeryFastTree`'s 376.54 s total runtime; 2.26 s and 4,947 wrong splits without exact full-profile acceptance verification), and at $N = 100{,}000$, it reduces wrong splits from 12,375 to **8**,**802** (**8.80**% nRF, beating `VeryFastTree` by **55** splits) in **104.45** s vs. +593.50 s (**5.7**× faster refinement, or **7.4**× faster than `VeryFastTree`'s 772.53 s total runtime; 3.46 s and 8,805 wrong splits without exact full-profile acceptance verification).

Table 10: Blind Empirical Benchmark on Biophysical RNASim 1,596-bp 16S rRNA Sequences (`bench_rnasim_gpu.cu`): 64-Thread `FastME 2.1.6` and `VeryFastTree 4.0.5` vs. GPU Phylo2Vec BME Optimizer ($N = 1{,}000 \ldots 100{,}000$).

Taxa (N)	Splits	VFT-InitNJ 64T	FastME 2.1.6 64T (SPR)	VFT-ME 64T (`-nom1`)	GPU Full-Tree BME (Time)	GPU Minibatch BME (Time)¶	Minibatch Speedup vs. VFT-ME	ΔL_{BME}
1,000	997	214 (21.46%, 0.58 s)	177 (17.75%, 21.68 s)†	178 (17.85%, +6.88 s)	**176** (**17.65**%, **0.01** s)‡	**176** (**17.65**%, **0.06** s)	**115**× (**361**× vs. FastME)	−0.19
5,000	4,997	890 (17.81%, 3.40 s)	> 35 *min (CPU wall)*	703 (14.07%, +33.38 s)	**692** (**13.85**%, 0.32 s)	696 (13.93%, **0.68** s)	**49**× (54× total)	−0.90
10,000	9,997	1,667 (16.68%, 7.82 s)	> 5 *hr (CPU wall)*	1,313 (13.13%, +65.98 s)	**1,309** (**13.09**%, 4.57 s)§	1,320 (13.20%, **1.55** s)	**43**× (48× total)	−1.63
50,000	49,997	6,729 (13.46%, 63.01 s)	*OOM / CPU wall*	4,980 (9.96%, +313.53 s)	*N/A (Minibatch)*	**4,950** (**9.90**%, **16.84** s)	**18.6**× (22.4× total)	−6.33
100,000	99,997	12,375 (12.38%, 179.03 s)	*OOM / CPU wall*	8,857 (8.86%, +593.50 s)	*N/A (Minibatch)*	**8,802** (**8.80**%, **104.45** s)	**5.7**× (7.4× total)	−11.35

†On raw gappy FASTA (`-d J -m B -n -s`), `FastME 64T` reaches 182 wrong splits (24.83 s); on GPU `JC69` matrix it reaches 177 (`BIONJ+SPR`, 21.68 s) and 178 (`-u InitNJ+SPR`, +11.27 s).

‡Both `JC69` and `Eigen-SymWLS` achieve 176/997 (17.65%) at $N = 1{,}000$; starting from a 100% random tree (997/997 wrong), GPU Full-Tree BME reaches 178/997 in 0.48 s.

§At $N = 10{,}000$, Full-Tree `Eigen-RevSymWLS` achieves **1**,**309** wrong splits (13.09%, 4.57 s), Full-Tree `JC69` achieves 1,320 (3.71 s), and Subtree-Wave Minibatch achieves 1,320 (13.20%, 1.55 s).

¶Sequential single-GPU wave execution with live outside-profile rebuilding and exact changed-pair uncapped acceptance verification (`k_refine_real_seq_condensed_D_pairs`); unverified parallel capped-proposal timings are 0.01 s, 0.22 s, 0.52 s, 2.26 s (4,947 RF), and 3.46 s (8,805 RF).

7 Conclusion

By formulating Balanced Minimum Evolution discrete gradients over the full $0 \leq j \leq 2i$ Phylo2Vec domain (Theorem 2.3), pairing them with $O(N)$ tree-based cherry-first gauge transformations (Theorem 2.1) and exact inverse bijections (Theorem 2.2), and using node-disjoint multi-coordinate (K-SPR) batch updates, Phylo2Vec optimization surpasses FastME in convergence speed by up

to 4.16 × –5.74× on a single CPU core while recovering identical or superior BME topologies across 99.2% (119/120) of tested datasets. Our Column-Independence (Theorem 4.1), Stackless Ancestor-Walk (Theorem 4.2), Heavy-First Logarithmic Register-Stack with race-free SIMT destination indexing (Theorem 4.3), Complete Column-Autonomy Tiling (Theorem 4.4), and Exact Clade-Condensation Minibatch (Theorem 4.5) reformulations convert pointer-chasing tree recurrences into lockstep, cache-resident matrix operations. They deliver up to a 9.01× `subdist` speedup (5.98× end-to-end hybrid speedup) on Apple M5 unified memory, up to a 62.5× single-GPU speedup on an NVIDIA RTX A6000 and a 112.8× single-GPU speedup at $N = 4{,}000$ on an NVIDIA H100 80GB HBM3, a 5.15× multi-GPU speedup (158.36 ms for 3.2×10^9 moves) at $N = 40{,}000$ on 8× H100, distributed column-sharded full-tree execution across 8× NVIDIA H100 up to $N = 380{,}000$ taxa (2.89×10^{11} moves in 205.45 s across 569.9 GB HBM3), clade-condensed minibatch optimization up to $N = 10{,}000{,}000$ taxa (39.20 s per full epoch on 8× H100 in 7.8 GB VRAM/GPU), and up to a 5.7 × –666× speedup over 64-thread `VeryFastTree 4.0.5` (and up to 4,087× over 64-thread `FastME 2.1.6`) on the 1,596-bp `JC69` and biophysical RNASim benchmarks while outperforming both tools in topological accuracy across every tested dataset size up to $N = 100{,}000$ (8,802 vs. 8,857 wrong splits in 104.45 s with exact full-profile acceptance verification vs. 772.53 s).

References

[1] Y. Pauplin. Direct calculation of a tree length using a distance matrix. *Journal of Molecular Evolution*, 51(1):41–47, 2000.

[2] R. Desper and O. Gascuel. Fast and accurate phylogeny reconstruction algorithms based on the minimum-evolution principle. *Journal of Computational Biology*, 9(5):687–705, 2002.

[3] V. Lefort, R. Desper, and O. Gascuel. FastME 2.0: A comprehensive, accurate, and fast distance-based phylogeny inference program. *Molecular Biology and Evolution*, 32(10):2798–2800, 2015.

[4] M. J. Penn, N. Scheidwasser, M. P. Khurana, D. A. Duchêne, C. A. Donnelly, and S. Bhatt. Phylo2Vec: a vector representation for binary trees. *Systematic Biology*, 73(6):871–886, 2024.

[5] S. Mirarab, N. Nguyen, S. Guo, L.-S. Wang, J. Kim, and T. Warnow. PASTA: Ultra-large multiple sequence alignment for nucleotide and amino-acid sequences. *Journal of Computational Biology*, 22(5):377–386, 2015.

[6] C. Piñeiro, J. M. Abuín, and J. C. Pichel. VeryFastTree: speeding up the estimation of phylogenies for large alignments through parallelization and vectorization strategies. *Bioinformatics*, 36(17):4658–4659, 2020.

[7] M. J. Penn, N. Scheidwasser, C. A. Donnelly, D. A. Duchêne, and S. Bhatt. Bayesian inference of phylogenetic distances: revisiting the eigenvalue approach. *Bulletin of Mathematical Biology*, 87:32, 2025.